

Vorlesung (WS 2016/17) *Softwarekonstruktion*

Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

Teil 3.2: Softwarearchitektur

v. 23.01.2016 und v. 30.01.2016

- Notation zum Beschreiben von Softwarearchitekturen
- Verständnis von ausgewählten Entwurfsprinzipien und Mustern von Softwarearchitekturen
- Übersicht über verschiedene Architekturstile

Was ist das Problem?

Softwarekonstruktion
WS 2016/17



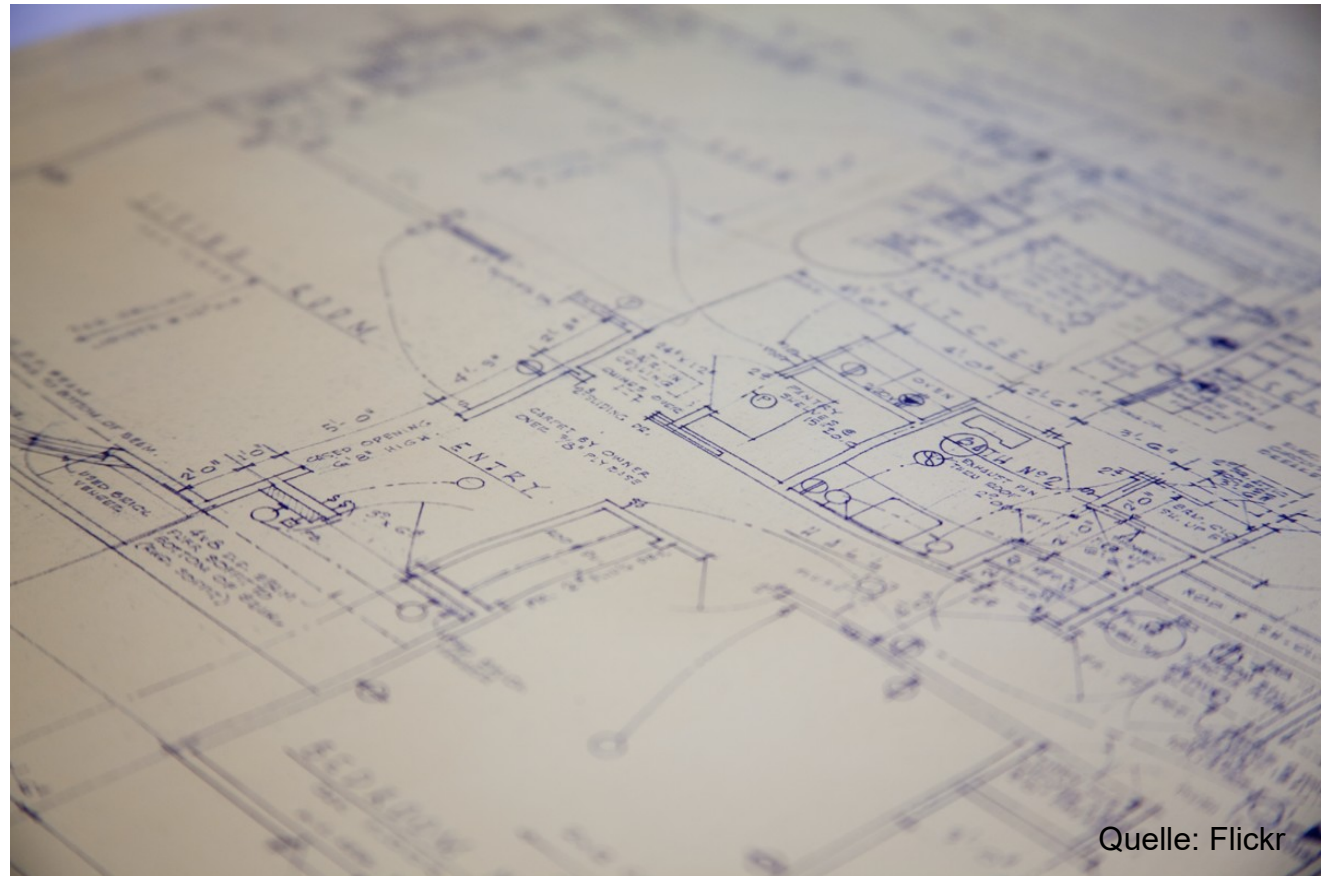
Quelle: Flickr

- Komplexe, verteilte Welt
- Offene Systeme
- Heterogene Umgebungen
- Mehr Brownfield- als Greenfield-Projekte
- Große Teams
- Qualität?



Quelle: Flickr

- Abstraktion des Systems mit reduzierten Details
- Dokument = Menge der Entwurfsentscheidungen?
(Medvidovic 2001)
 - Kommunikation
 - Vertrag
 - Rahmenwerk für Entwickler



Quelle: Flickr

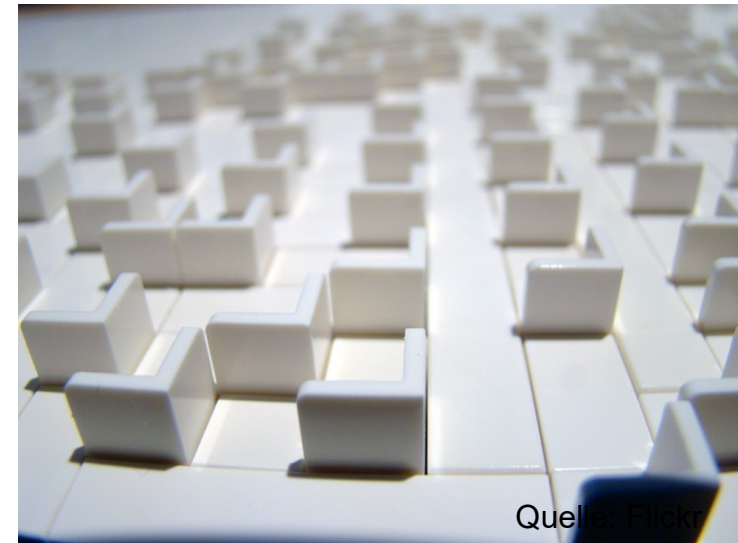
Eine **Softwarearchitektur** (SA) definiert:

- die Komponenten eines Softwaresystems,
- wie die Komponenten untereinander Funktionalität und Daten bereitstellen,
- wie die Kontrolle zwischen den Komponenten geregelt ist.

Höchste Entwurfsebene – Lösungsstruktur im großen Maßstab

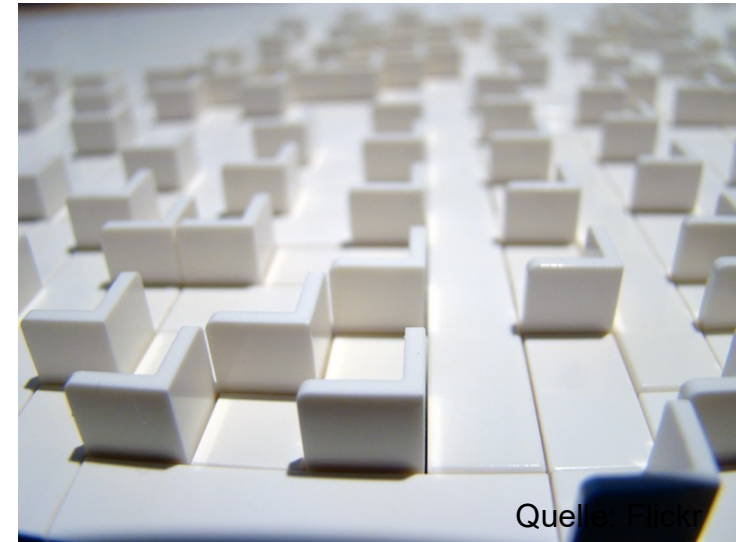
SA ist Teil des Softwareentwurfs

Was sind Beispiele von Komponenten in Softwarearchitekturen?



Was sind Beispiele von Komponenten in Softwarearchitekturen?

- Klassen/Objekte (z.B. Java Klasse)
- Module (z.B. Dienste)
- (Sub-)Systeme (z.B. E-Mail Server oder Internet)



Funktionale Anforderung legt fest,
was das Produkt tun soll

Nichtfunktionale Anforderungen sind genereller

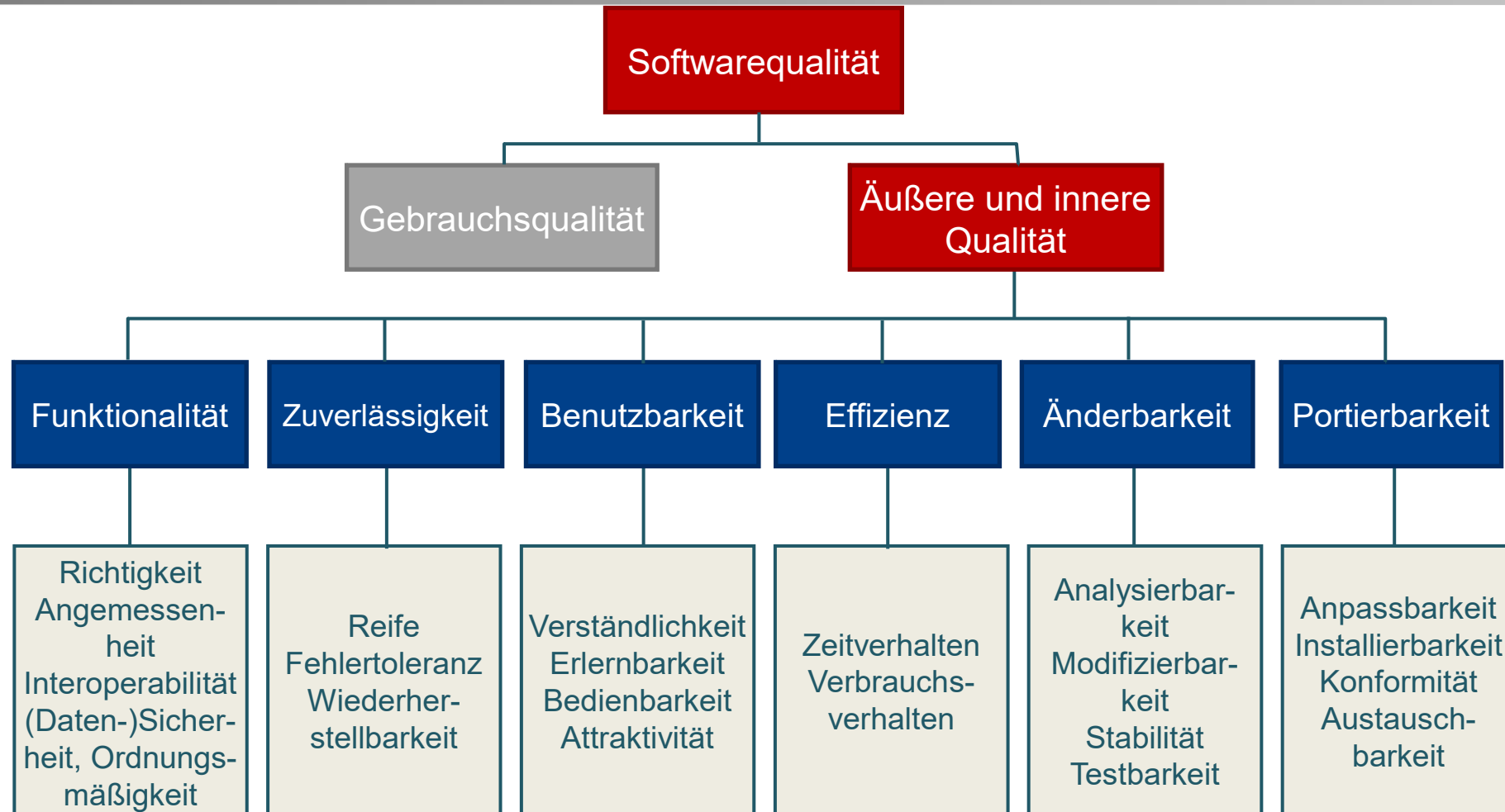
Welche nichtfunktionalen Anforderungen gibt es?

- Zuverlässigkeit: Systemreife, Wiederherstellbarkeit, Fehlertoleranz, Robustheit
- Aussehen und Handhabung: Look and Feel
- Benutzbarkeit: Verständlichkeit, Erlernbarkeit, Bedienbarkeit
- Leistung und Effizienz: Antwortzeiten, Ressourcenbedarf, Wirtschaftlichkeit
- Betrieb und Umgebungsbedingungen
- Wartbarkeit, Änderbarkeit: Analysierbarkeit, Stabilität, Prüfbarkeit, Erweiterbarkeit

- Portierbarkeit und Übertragbarkeit: Anpassbarkeit, Installierbarkeit, Konformität, Austauschbarkeit
- Sicherheitsanforderungen: Vertraulichkeit, Informationssicherheit, Datenintegrität, Verfügbarkeit
- Korrektheit: Ergebnisse fehlerfrei
- Flexibilität: Unterstützung von Standards)
- Skalierbarkeit: Änderungen des Problemumfangs bewältigen
- Randbedingungen

FURPS:

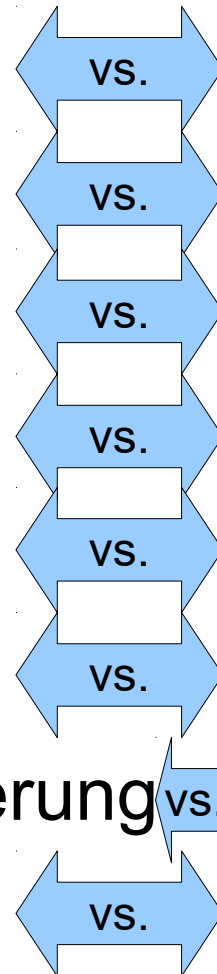
- Functionality (Funktionalität)
- Usability (Benutzbarkeit)
- Reliability (Zuverlässigkeit)
- Performance (Effizienz)
- Supportability (Änderbarkeit/Wartbarkeit)



ISO/IEC 9126: Bewerten von Softwareprodukten, Qualitätsmerkmale und Leitfaden zu ihrer Verwendung. (Seit 2005 ersetzt durch ISO/IEC 25000 Software engineering – Software product Quality Requirements and Evaluation (SQuaRE).)

Form

- Interne Anforderungen
- Strenge Kontrolle
- Geld- und Zeitkosten
- Komplexität
- Neue Technologien
- Top-Down-Planung
- Fortlaufende Verbesserung
- Knappe Integration



Funktion

- Externe Anforderungen
- Flexible Änderungen
- Zusätzliche Funktionen
- Verständlichkeit
- Bewährte Technologien
- Bottom-Up-Planung
- Stabilität
- Wenige Interfaces

Aufgabe des Softwarearchitekten?

Aufgabe des Softwarearchitekten?

- Umsetzbaren Entwurf entwickeln der Anforderungen erfüllt
- Findung, Beschreibung und Begründung der für die Umsetzung gewählten technischen Ansätze
- Balance der Anforderungen erreichen
- Verständlichen Entwurf, der Entwurfsentscheidungen nachvollziehbar macht

Arten: Enterprise-Architekt, Solutions-Architekt und Applikationsarchitekt

Subsysteme

- Ersetzbare Teile mit wohldefinierten Schnittstellen (interfaces), welche den Zustand und Verhalten von beinhalteten Komponenten oder Klassen kapseln.
- UML Darstellungsmöglichkeiten:
 - Komponenten-Diagramme
 - Paket-Diagramme
 - Verteilungs-Diagramme

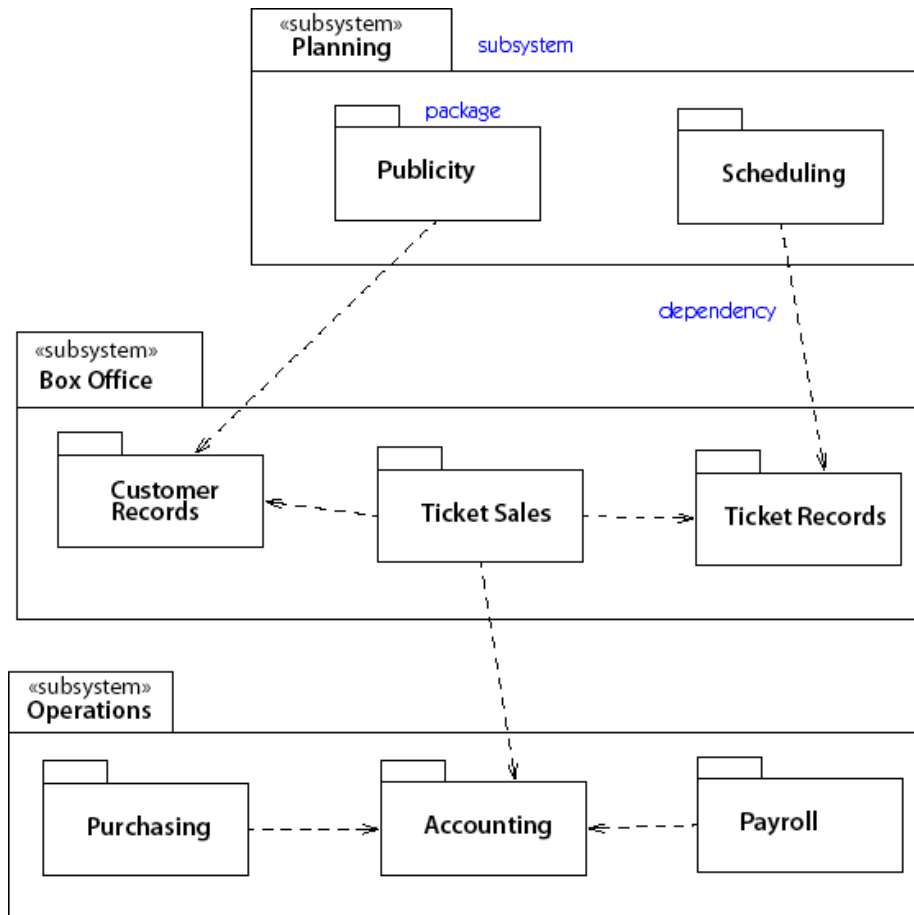


Figure 3-10. Packages

Pakete
Hierarchien
Abhängigkeiten

Nützlich um Abhängigkeiten von
Hauptelementen im System auszudrücken

Physikalische Verteilung von Laufzeitkomponenten auf Hardwareknoten

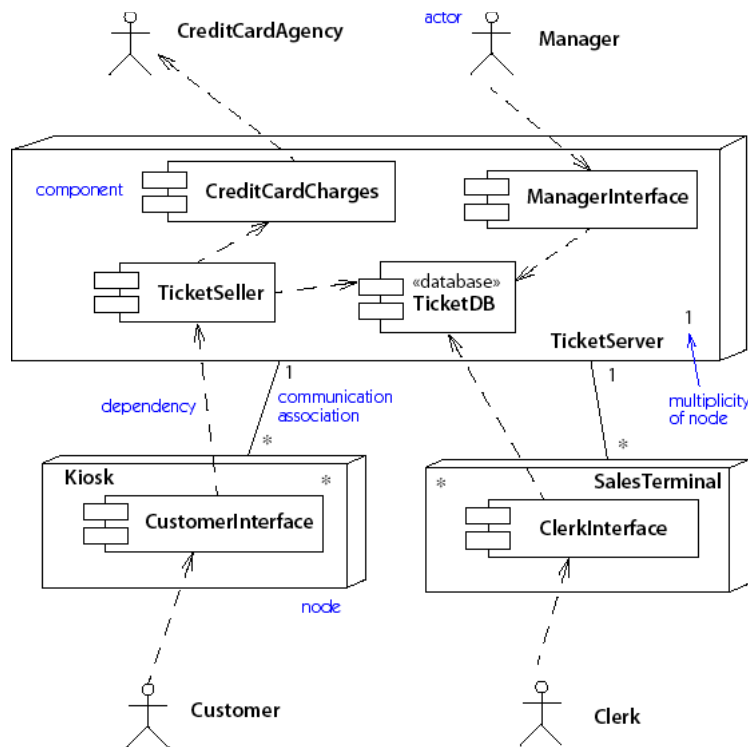


Figure 3-8. Deployment diagram (descriptor level)

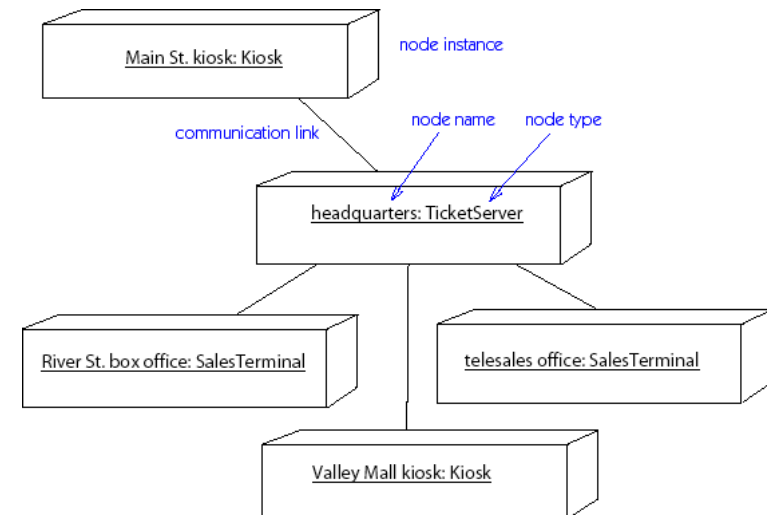


Figure 3-9. Deployment diagram (instance level)

- Komponenten:** Menge von verwandten Operationen mit einem gemeinsamen Verwendungszweck in einer unabhängigen auslieferbaren Einheit.
- Modul:** Systemkomponente, welche Dienste für andere Komponenten bereitstellt, jedoch nicht als separates System zu betrachten ist.
- Schnittstellen:** Menge von Operationen, welche anderen Subsystemen zur Verfügung stehen

Qualitäten von Beziehungen

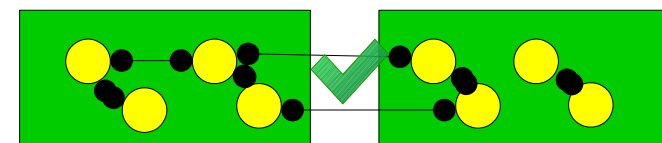
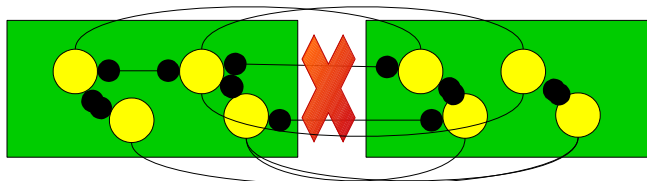
Kopplung (coupling) : Menge von Beziehungen zwischen Subsystemen

- Lose Kopplung
- allgemeines Entwurfsziel:
 - Unabhängigkeit
 - Instandhaltungsvermögen / Wartbarkeit
- Kann zusätzliche Abstraktionsstufen bedürfen

Kohäsion (cohesion): Menge von Beziehungen innerhalb eines Subsystems

- Hohe Kohäsion
- allgemeines Entwurfsziel:
 - Reduzierte Komplexität
- Häufig Zielkonflikt

- **Minimieren** der Kopplung zwischen Modulen
 - Ziel: Module müssen wenig über Interaktion mit anderen Modulen wissen
 - Lose Kopplung erleichtert zukünftige Änderungen
- **Maximieren** der Kohäsion in Modulen
 - Ziel: Inhalte jeder Komponente sollten stark zusammenhängend sein
 - Hohe Kohäsion erleichtert das Verstehen einer Komponente



Problem:

Wie unterstützt man geringe Abhängigkeit, geringe Auswirkung von Änderungen und hohe Wiederverwendbarkeit?

- **Maß**, wie stark ein Element verbunden ist, besitzt Wissen über andere Elemente oder hängt von anderen Elementen ab.
- Element mit loser Kopplung **hängt nicht** von vielen anderen Element ab.

Typische Formen der Kopplung zwischen TypX und TypY:

- TypX besitzt Attribut mit Verweis auf Instanz von TypY
- TypX Objekt ruft Dienst eines TypY Objekts auf
- TypX hat Methode welche TypY Instanz referenziert
 - z.B. Parameter, lokale Variable, Rückgabewert
- TypX ist direkte oder indirekte Subklasse von TypY
- TypY ist Schnittstelle und TypX implementiert diese Schnittstelle

Klasse mit hoher oder starker Kopplung ist auf viele andere Klassen angewiesen. Solche Klassen nicht wünschenswert und haben folgende Probleme:

- Lokale Änderungen notwendig bei Änderungen in verbundenen Klassen
- Isoliertes Verstehen der Klassen schwierig (big picture)
- Geringere Wiederverwendbarkeit, da Klassenverwendung Anwesenheit weiterer verbundener Klassen benötigt

Lösung:

Klasse mit hoher oder starker Kopplung ist auf viele andere Klassen angewiesen. Solche Klassen nicht wünschenswert und haben folgende Probleme:

- Lokale Änderungen notwendig bei Änderungen in verbundenen Klassen
- Isoliertes Verstehen der Klassen schwierig (big picture)
- Geringere Wiederverwendbarkeit, da Klassenverwendung Anwesenheit weiterer verbundener Klassen benötigt

Lösung:

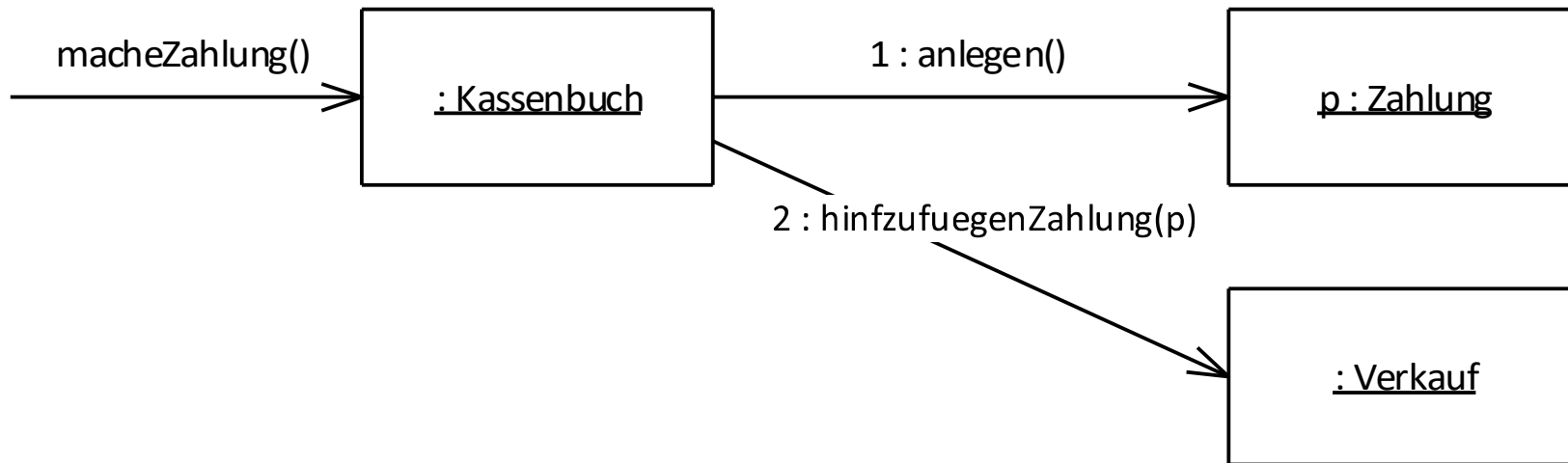
- Verantwortlichkeiten zuweisen für lose Koppelung.
- Dieses Prinzip hilft, alternative Lösungen zu finden.

Beispiel für Lösungsbewertung durch loose Koppelung

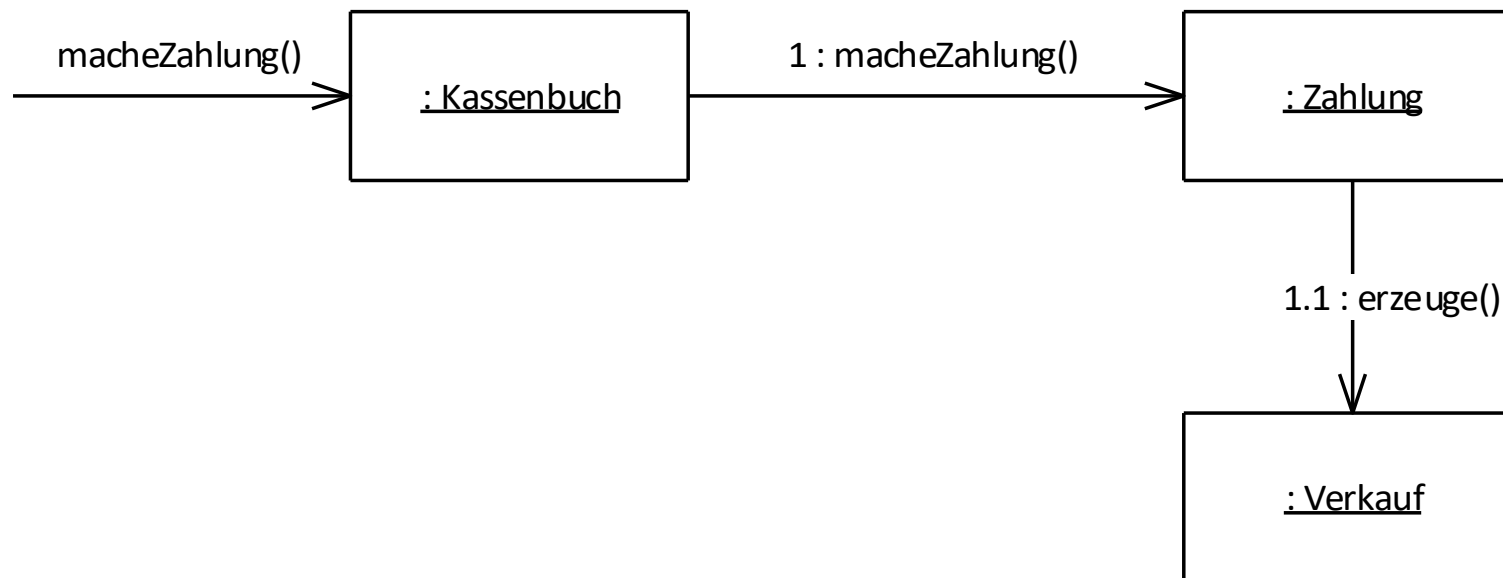
Gegeben seien folgende Klassen

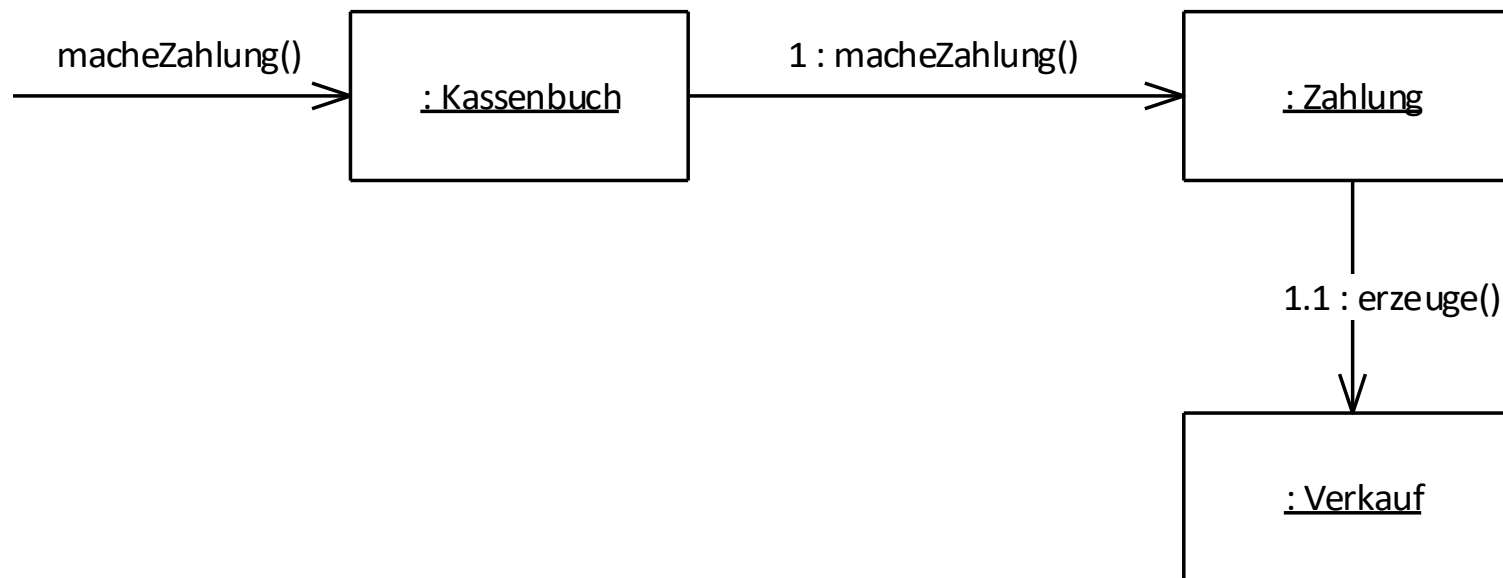


- Instanz „Zahlung“ muss erzeugt werden und mit Verkauf verbunden werden.
- Welche Klasse ist die Verantwortliche?



Diese Lösung ist Erzeuger-Muster (creator pattern)





Welche Lösung ist besser?

Angenommen jeder *Verkauf* ist mit *Zahlung* verbunden.

Lösung #1 besitzt Kopplung zwischen *Kassenbuch* und *Zahlung*, welche nicht in Lösung #2 vorkommt.

Lösung #2 besitzt daher geringere Kopplung.

Bemerke: Muster von loser Kopplung und Erzeuger liefern unterschiedliche Lösungen

Niemals Muster isoliert betrachten!

- Lose Kopplung ermöglicht Entwürfe mit hoher Unabhängigkeit, was die Auswirkungen von Änderungen verringert.
- Muss mit anderen Mustern wie Informationsexperte oder hohe Kohäsion betrachtet werden.
- Implementierungsvererbung erhöht Kopplung – insbesondere zwischen Domänenobjekten welche von technischen Diensten erben (z.B. PersistentObject)
- Hohe Kopplung mit stabilen, globalen Objekten ist unproblematisch (z.B. Java Bibliothek java.util)

Hohe Kopplung per se kein Problem. Hohe Kopplung zu unbeständigen Elementen schon.

Entwickler können mittels loser Kopplung und Kapselung diverse Systemaspekte zukunftssicher machen (braucht aber eine gute Begründung).

Fokussieren auf ausgewählte Punkte mit realistischer hoher Instabilität und Evolution.

Wie bündelt man ein Objekt, macht es verständlich, handhabbar und erreicht – ganz nebenbei – eine lose Kopplung?

Wie bündelt man ein Objekt, macht es verständlich, handhabbar und erreicht – ganz nebenbei – eine lose Kopplung?

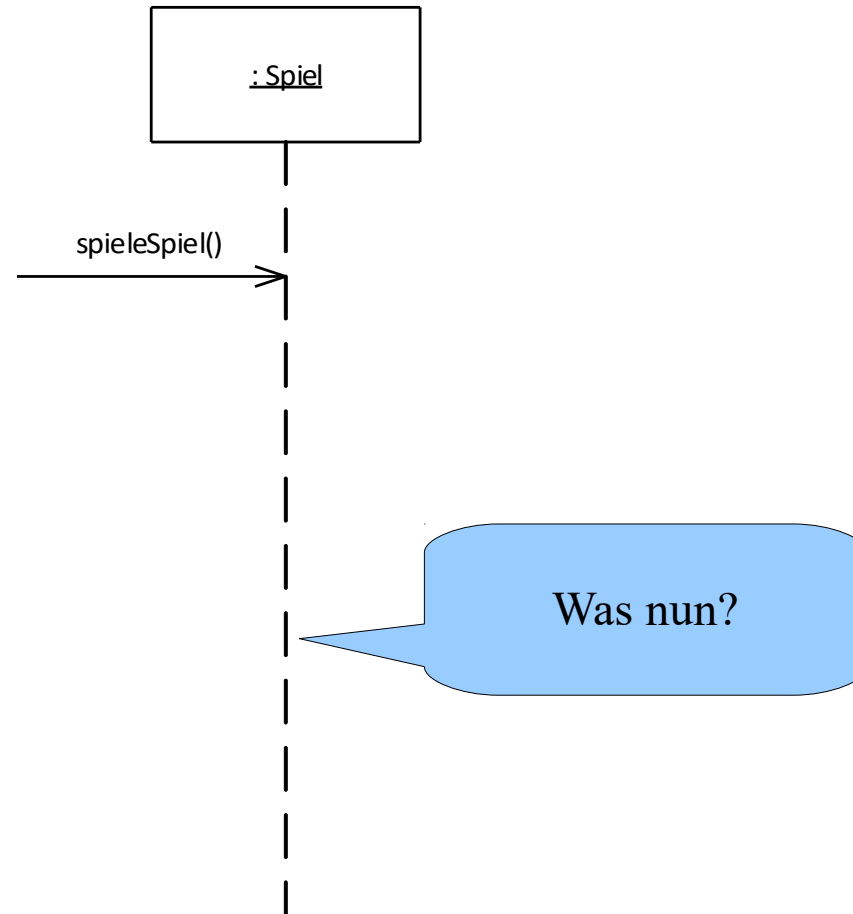
Kohäsion ist Maß, wie stark zusammenhängend und gebündelt die Verantwortlichkeiten im Element sind.

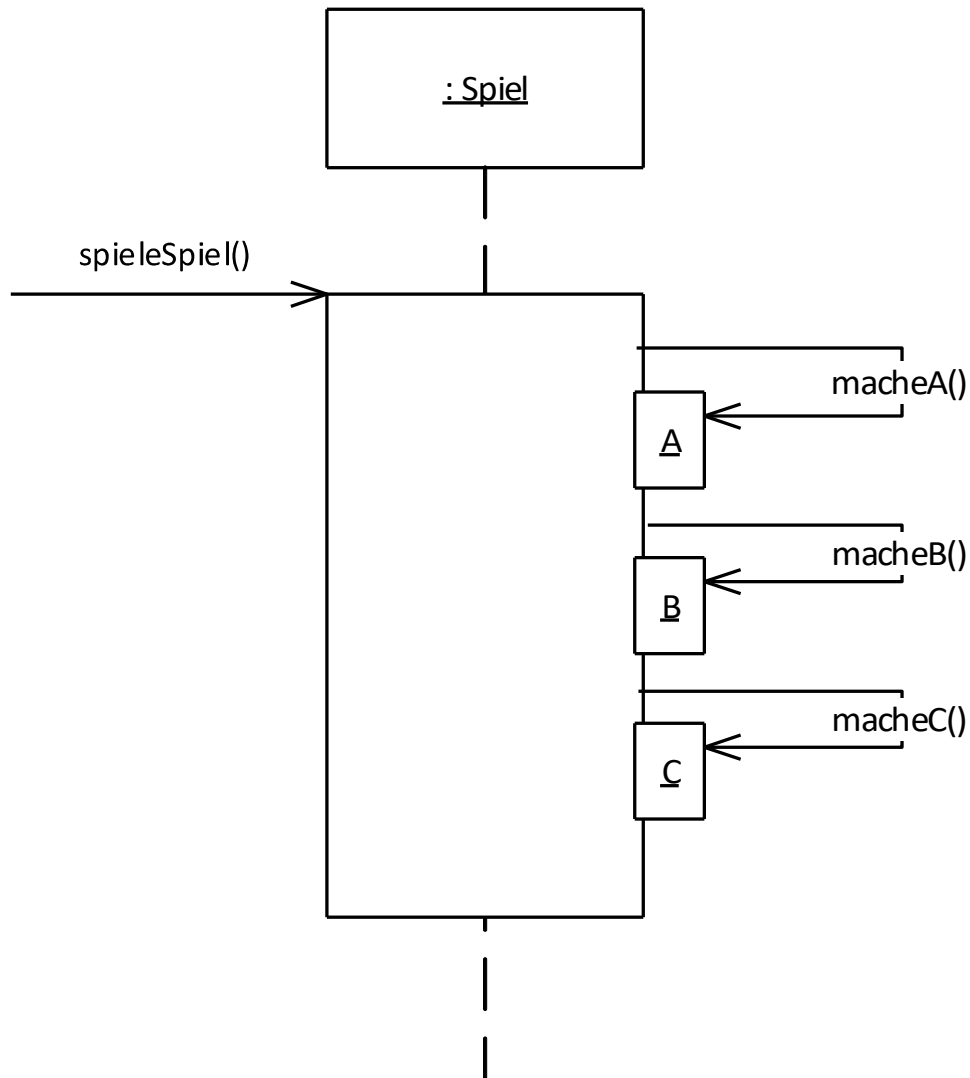
Element mit hohen verbundenen Verantwortlichkeiten, das nicht viel Funktionalität bietet, hat eine hohe Kohäsion.

Element mit geringer Kohäsion bietet verschiedene, unzusammenhängende Funktionalitäten oder wenig Funktionalität. Solche Elemente sind unerwünscht und erzeugen folgende Probleme:

- schwer verständlich
- schwer wiederzuverwenden
- schwer zu warten
- anfällig, andauernd von Änderungen beeinflusst

- Verantwortlichkeiten so zuordnen, dass Kohäsion hoch bleibt
- Lösung anwenden, um Alternativen zu generieren

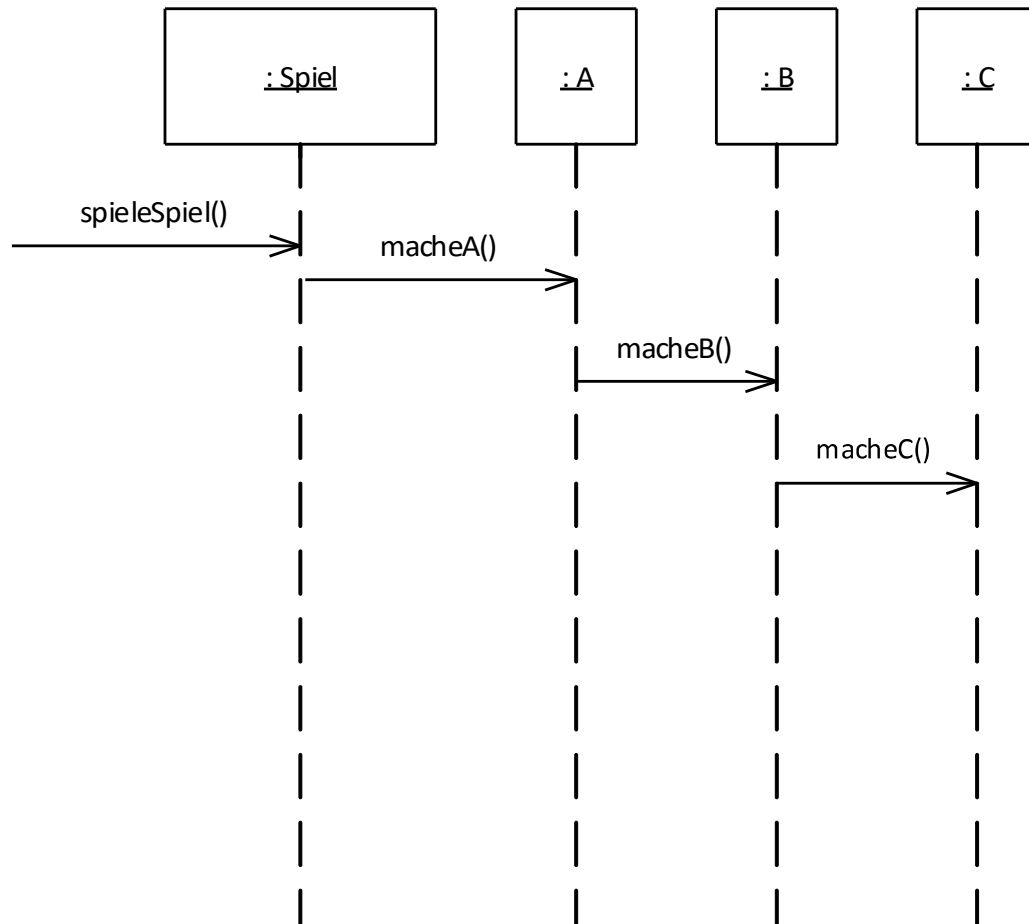




Object Spiel macht alles.

Wie hängen Aufgaben A, B und C zusammen?

Falls geringe Abhängigkeit,
dann auch geringe Kohäsion!



Delegates verteilen Aufgabe, voraussichtlich an Objekte mit passend zugeordneter Verantwortlichkeit

- **Sehr niedrige Kohäsion:** Klasse bietet vollständig unterschiedliche Funktionalität, z.B. Datenbankverbindung und RPC.
- **Niedrige Kohäsion:** Klasse hat eine Verantwortlichkeit für eine komplexe Aufgabe in einer komplexen Funktionalität, z.B. eine einzelne Schnittstelle für alle Datenbankzugriffe.
- **Mittlere Kohäsion:** Leichtgewichtige Klasse verantwortlich für einige wenige logische Bereiche, z.B. *Unternehmen* kennt Mitarbeiter und Finanzdetails.
- **Hohe Kohäsion:** Klasse mit moderater Verantwortlichkeit in einem funktionalen Bereich, welche mit anderen Klassen zusammenarbeitet.

Um hohe Kohäsion aufzuweisen, muss eine Klasse

- wenige Methoden besitzen,
- wenige Codezeilen groß sein,
- nicht zu viel Funktionalität bieten,
- eine hohe Abhängigkeit des Programmcodes besitzen.

Teil II: Architekturstile und -muster

Architekturstile

An **architectural style** defines a **family of systems** in terms of a pattern of structural organization. More specifically, an architectural style defines a vocabulary of **components** and **connector** types, and a set of **constraints** on how they can be combined.

- Shaw and Garlan

- Objekt-orientiert
- Client-Server, Object-Broker, Peer to peer
- Pipe and filter
- Ereignis-orientiert – publish/subscribe
- Schichten – drei-/vier-Schicht Architektur
- Repository – blackboard, model/view/controller (MVC)
- Process control

Beispiel

- Abstrakte Datentypen (Module)

Wichtige Eigenschaften

- Kapselung (interne Datendarstellung von außen nicht sichtbar)
- Kann Problem in Menge von interagierenden Agenten zerlegen
- Kann multi-threaded oder single-threaded sein

Nachteil

- Objekte müssen Identität anderer Objekte kennen, um zu interagieren

Eine **Client-Server Architektur** verteilt Anwendungslogik und -funktionalität auf eine Menge von Clients und Server Subsysteme, welche u.U. auf verschiedenen Maschinen ausgeführt werden und über Netzwerk kommunizieren.

- Wichtige Eigenschaften
 - Verteilung der Daten einfach
 - Client muss andere Clients nicht kennen
 - Unterteilt ein System in handhabbare Teile
 - Unabhängiger Kontrollfluss
 - Server meistens verantwortlich für Persistenz und Konsistenz von Daten

Nachteil

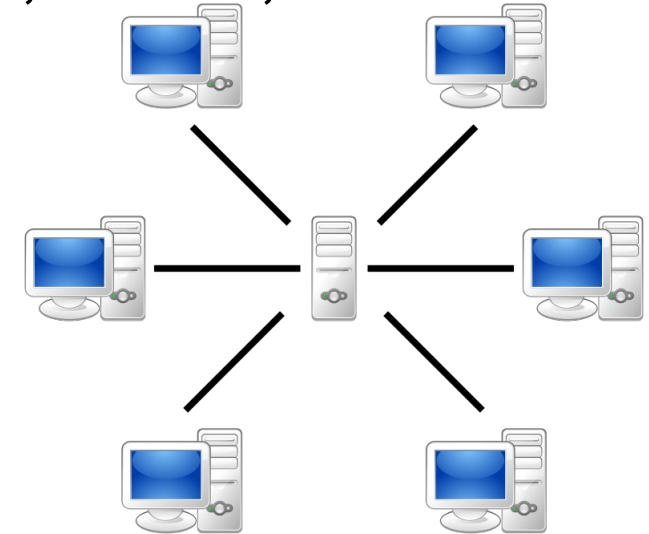
- Client muss Identität des Servers kennen (zentrales Verzeichnis)
- Kein gemeinsames Datenmodell
- Redundantes Management der Server

Client/Server kommunizieren z.B. RPC, SOAP, REST

Verteilte Systeme, z.B. Web Services,
Systeme mit zentraler DB

Varianten

- Fette Clients mit eigenen Diensten
- Leichte Clients ohne eigene Dienste



Quelle: wikipedia.org

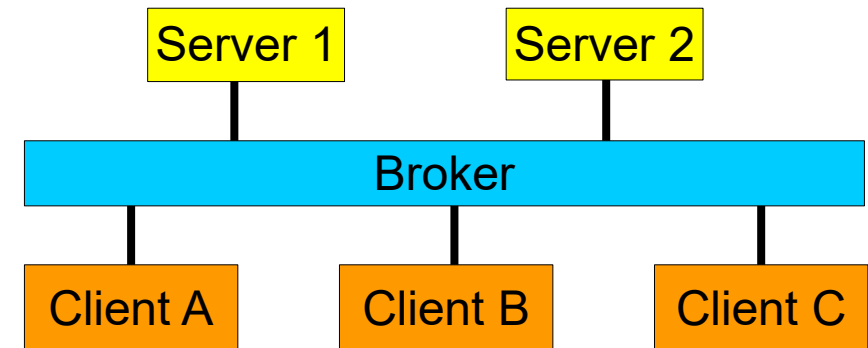
53

Wichtige Eigenschaften

- Broker als Indirektion zwischen Client und Server
- Clients müssen Server nicht mehr kennen
- Mehrere Broker und mehrere Server möglich

Nachteile:

- Broker sind Engpässe und Single-Point-of-Failure
- Verminderte Performanz durch Indirektion

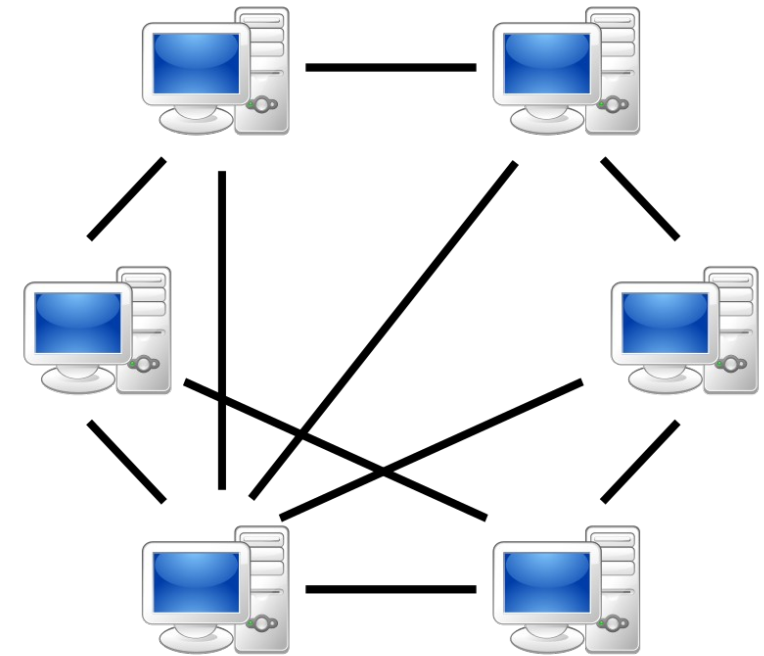


Wichtige Eigenschaften

- Findet Peers durch Server oder Broadcast
- Kommuniziert nachher nur mit Peers
- Reduziert Engpässe und robust gegenüber Peer Ausfällen

Nachteile

- Server kann Engpass werden
- Peers haben unvollständige Sicht – Synchronisation möglich



Quelle: wikipedia.org

55

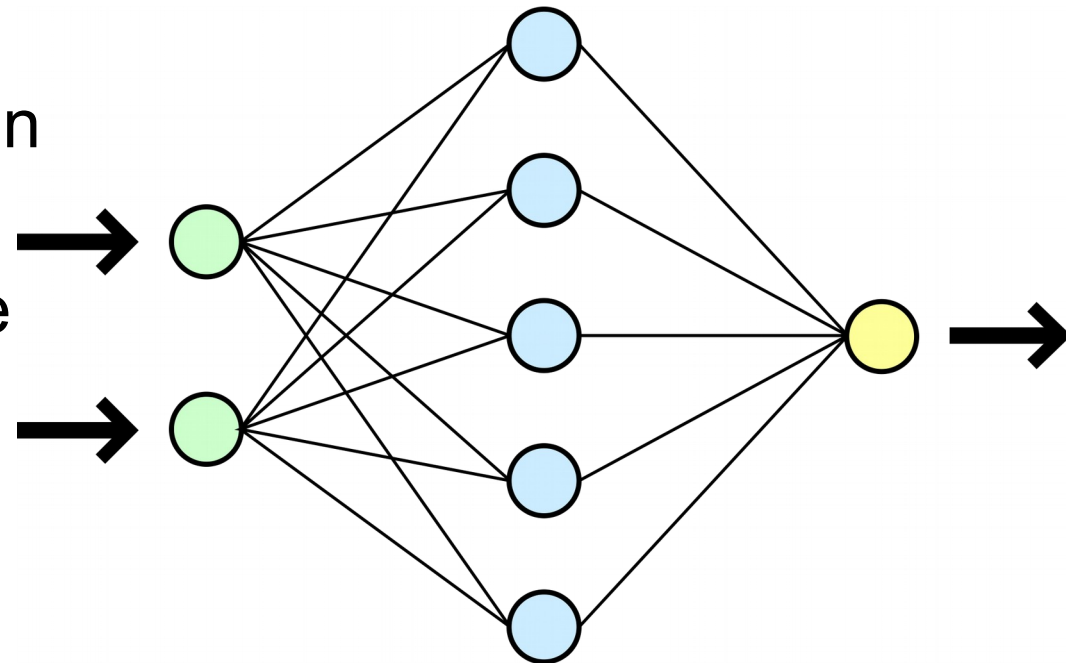
Komponenten in **Datenfluss Architektur** führen funktionale Transformation auf Eingaben aus und erzeugen Ausgaben.

- Sehr effektive Methode für Latenzreduktion in parallel oder verteilten Systemen

- keine Aufruf/Antwort Kosten
- wegen Sequentialisierung müssen schnelle Prozesse auf langsame warten

Nachteil: Nicht geeignet für interaktive Systeme

- Datenflüsse sind schleifenfrei



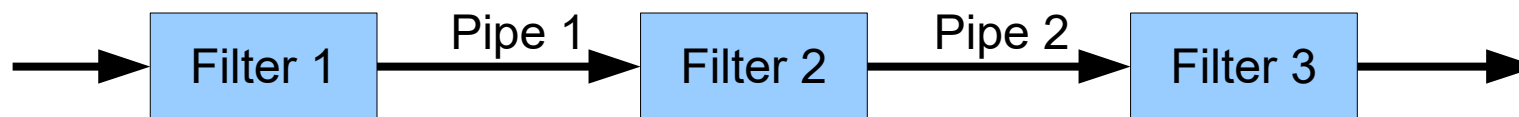
Quelle: wikipedia.org

Beispiele

- Unix-Pipes
- Compiler Kette: Lexer->Parser->Semantische Analyse-> ...
- Signal Prozessoren

Wichtige Eigenschaften

- Filter müssen nichts über verbundene Elemente wissen
- Filter können parallel implementiert sein
- Systemverhalten = Komposition der Filter
 - spezielle Analysen wie Durchsatz oder Deadlocks möglich



Komponenten in **Ereignis-basierte Architektur** liefern Dienste als Reaktion auf externe Ereignisse von anderen Komponenten.

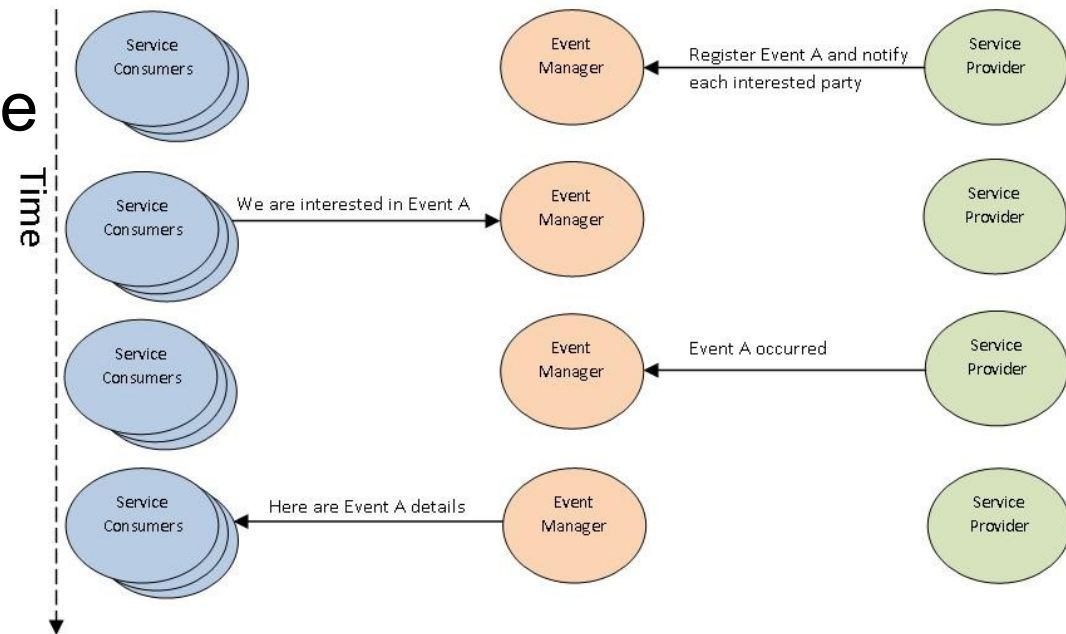
- Beispiele
 - Debugger (lauschen nach speziellen Breakpoints)
 - DBMS (für Integritätschecks)
 - Graphische Benutzerschnittstellen
- Im **Broadcast Modell** werden Ereignisse an alle Subsysteme gesendet. Jedes Subsystem kann Ereignis behandeln.
- Im **Interrupt-Modell** wird Echtzeit-Interrupt von einem Handler aufgefangen und an andere Komponente weitergeleitet.

Wichtige Eigenschaften

- Ereignisquelle muss Ereignissenken nicht kennen (impliziter Aufruf)
- Unterstützt Wiederverwendung und Evolution

Nachteil

- Komponenten haben keine Kontrolle von Berechnungsreihenfolge



Schichtenarchitektur organisiert System in Menge von Schichten, wobei jede Schicht der darüberliegenden Schicht Funktionalität zur Verfügung stellt.

- Beispiele
 - Betriebssysteme
 - Kommunikationsprotokolle

Wichtige Eigenschaften

- Normale Schichten beschränkt, da Element nur:
 - andere Elemente in gleichen Schicht oder
 - Elemente aus darunterliegenden Schicht sehen
- Unterstützen wachsende Abstraktion während Entwurf

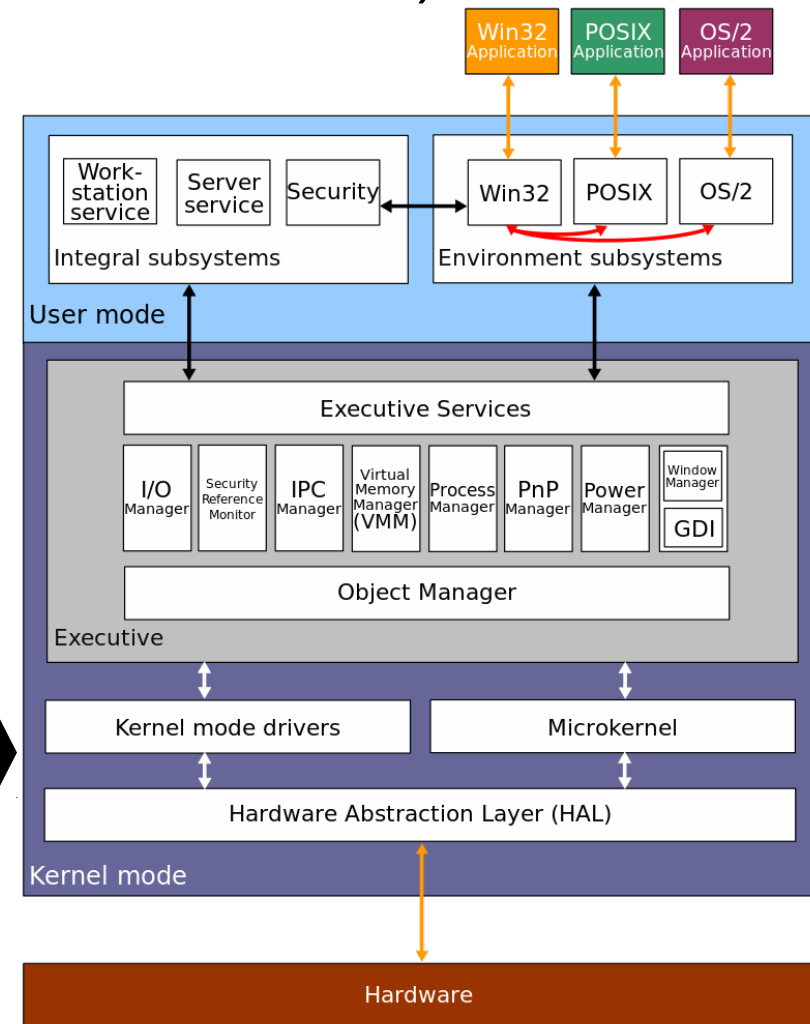
- Erlauben Erweiterungen (additive Funktionalität) und Wiederverwendbarkeit

- Möglichkeit Standardschicht Schnittstellen zu definieren

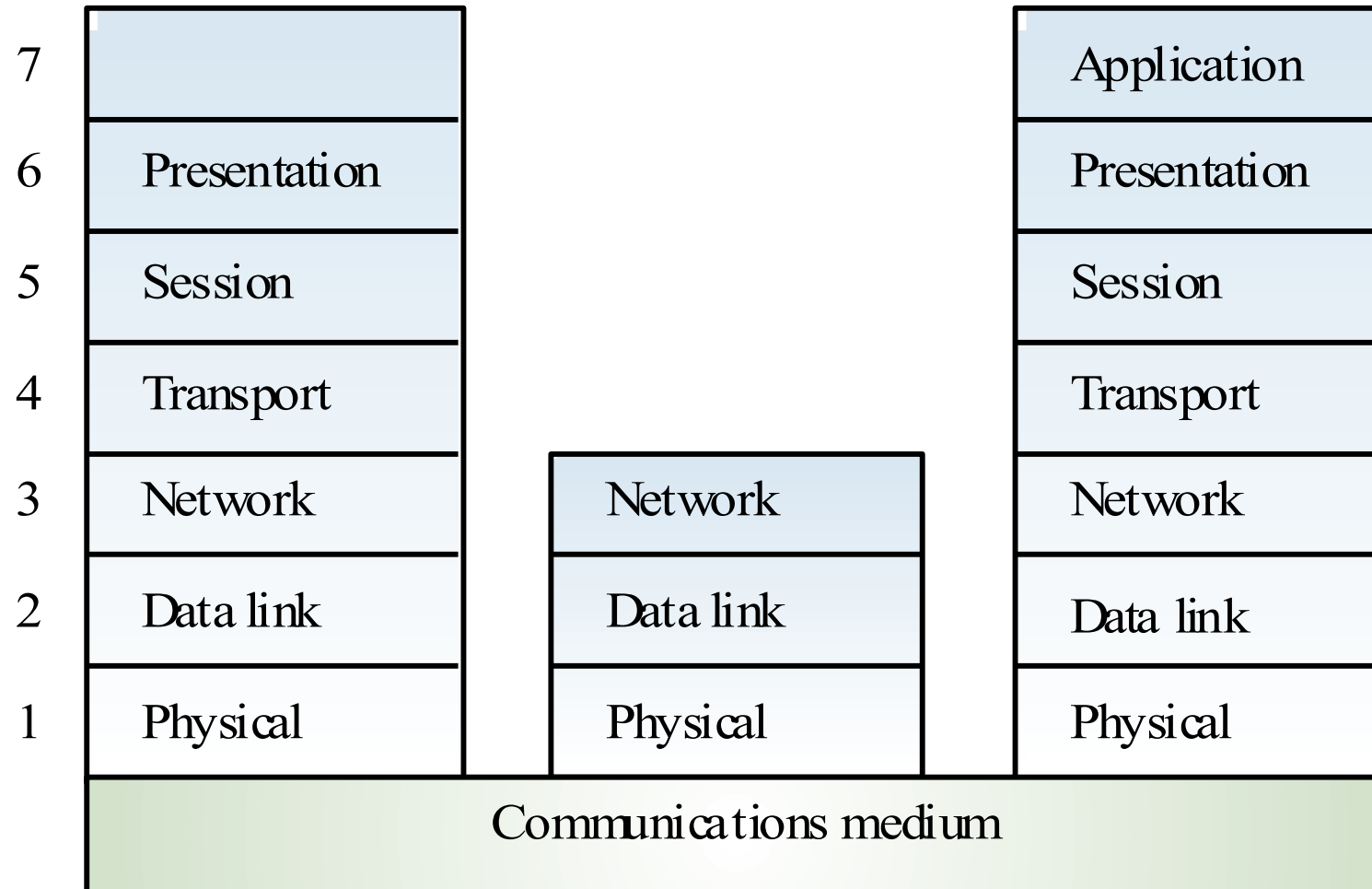
- Callbacks zur Kommunikation

Nachteil

- Schwierig saubere Schichttrennungen zu definieren
- Darstellung in Layer Diagramm ➡
Beispiel: MS Windows 2000 OS



Quelle: MS Windows 2000 OS, wikimedia.org

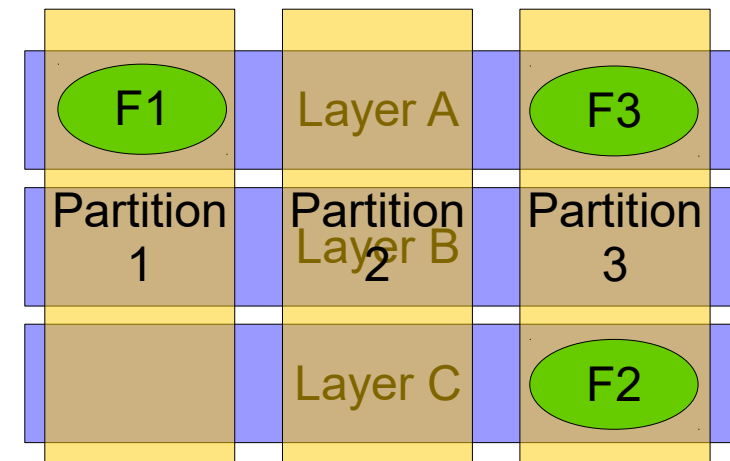


Schichten

- Hierarchische Dekomposition → Baum von Subsystemen
- „Horizontale“ Unterteilung
- Offen vs. geschlossene Schichten

Zerlegung (Partition)

- „Vertikale“ Unterteilung: Jede Teilmenge behandelt eine Funktion
- Extremform der Entkoppelung: semi-unabhängig



Geschlossene Architektur

- Jede Schicht verwendet ausschließlich Dienste der direkt darunterliegenden Schicht
- Minimiert Abhängigkeiten zwischen den Schichten und reduziert Auswirkungen von Veränderungen

Offene Architektur

- Eine Ebene nutzt Dienste von darunterliegenden Schichten
- Kompakterer Code, da darunterliegende Services direkt zugreifbar sind
- Weicht Kapselung der Schichten auf und erhöht Abhängigkeiten

Beispiele

- Datenbanken
- Blackboard Expertensysteme
- Programmierumgebungen

Wichtige Eigenschaften

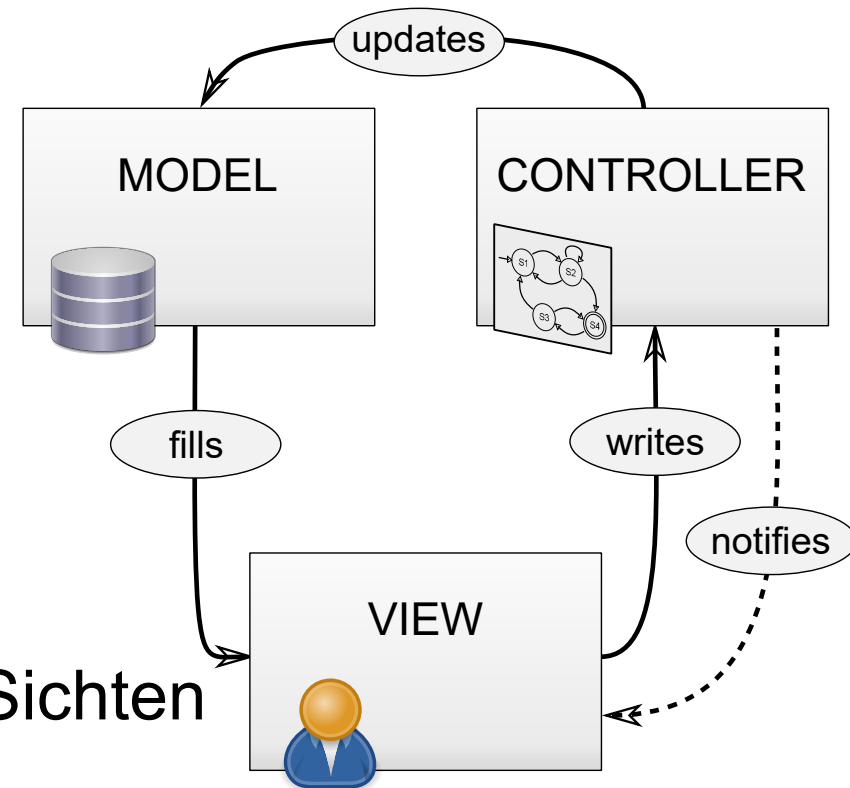
- Ort der Kontrolle wählbar (Agenten, Blackboard, beides)
- Reduziert Notwendigkeit komplexe Daten zu duplizieren

Nachteil

- Blackboard wird Engpass

Blackboard Architektur (Repository) verteilt Anwendungs-logik auf unabhängige Subsysteme, verwaltet jedoch alle Daten zentral in **einzelnem, gemeinsamen Repository**

- Subsysteme greifen und modifizieren einzelne Datenstrukturen
- Effiziente Möglichkeit für komplexe, wandelbare Daten
- Nebenläufigkeit und Datenkonsistenz
- Kontrolle durch Subsystem oder Blackboard
- Nachteil: Möglicher Engpass und reduzierte Modifizierbarkeit
- Beispiele: DBs, IDE's, tuple spaces



Wichtige Eigenschaften:

- Ein zentrales Modell und mehrere Sichten
- Jede Sicht hat eigenen Controller
- Controller verarbeitet Updates der Nutzer (Sicht)
- Controller ändert Modell und propagiert Änderungen an Sichten

Beispiel

- Flugsteuerungssysteme
- Controller für industrielle Fertigungsanlagen

Wichtige Eigenschaften

- Trennt Kontrollpolitik vom kontrollierten Prozess
- Behandelt Echtzeit und reaktive Berechnungen

Nachteil

- Zeitvorgaben schwer zu spezifizieren und Antwort auf Störungen

Moderne Architekturstile

REST

Representational State Transfer

REST-Paradigma von Roy Fielding 1994 als rationale Rekonstruktion des HTTP Object Models: GET, POST, ...

Seit 2000 bekannt als REST-Architekturstil

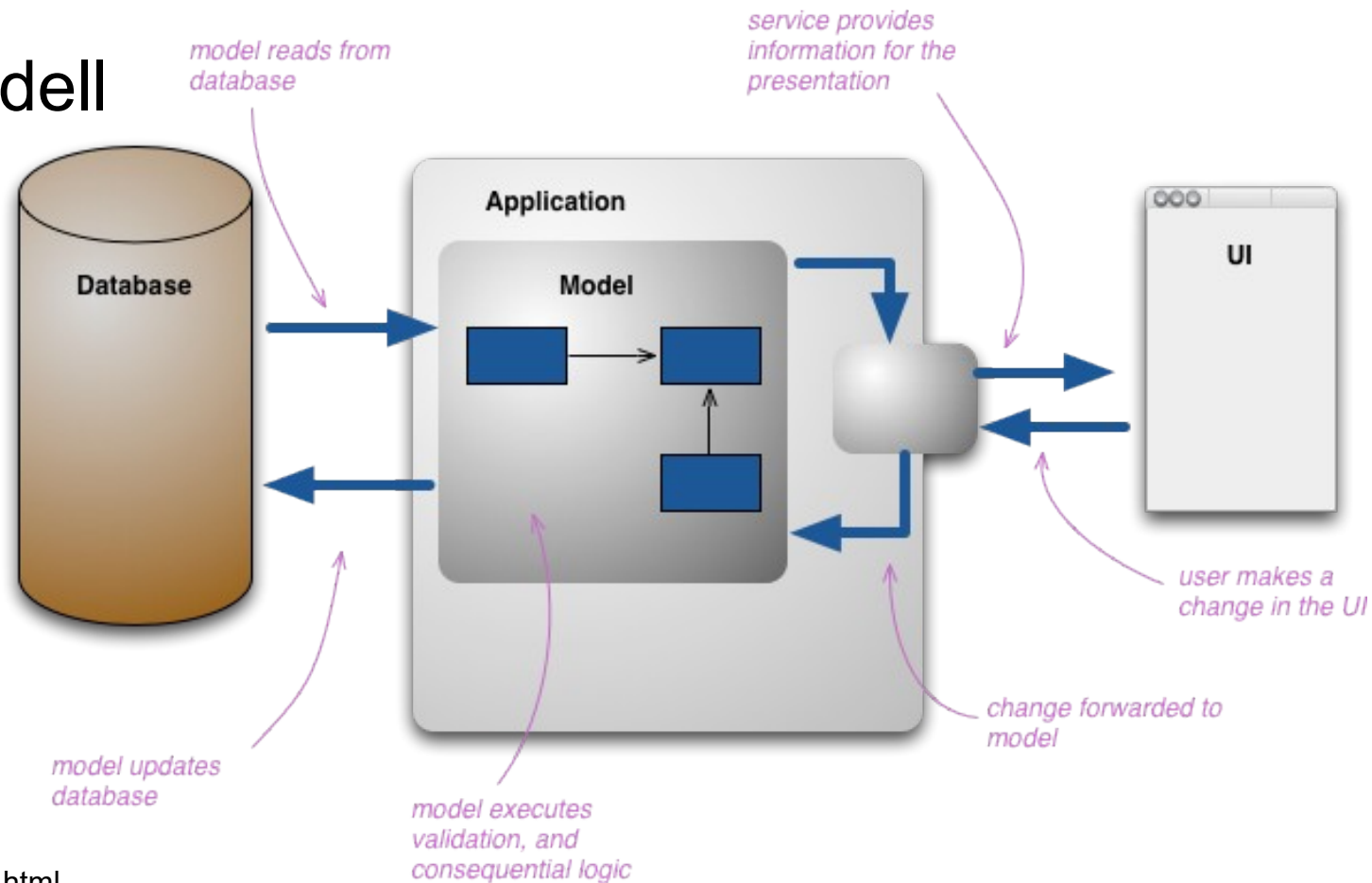
Prinzipien

- Client-Server
- Zustandslosigkeit
- Caching
- Einheitliche Schnittstelle
- Mehrschichtige Systeme
- Code on Demand (optional)



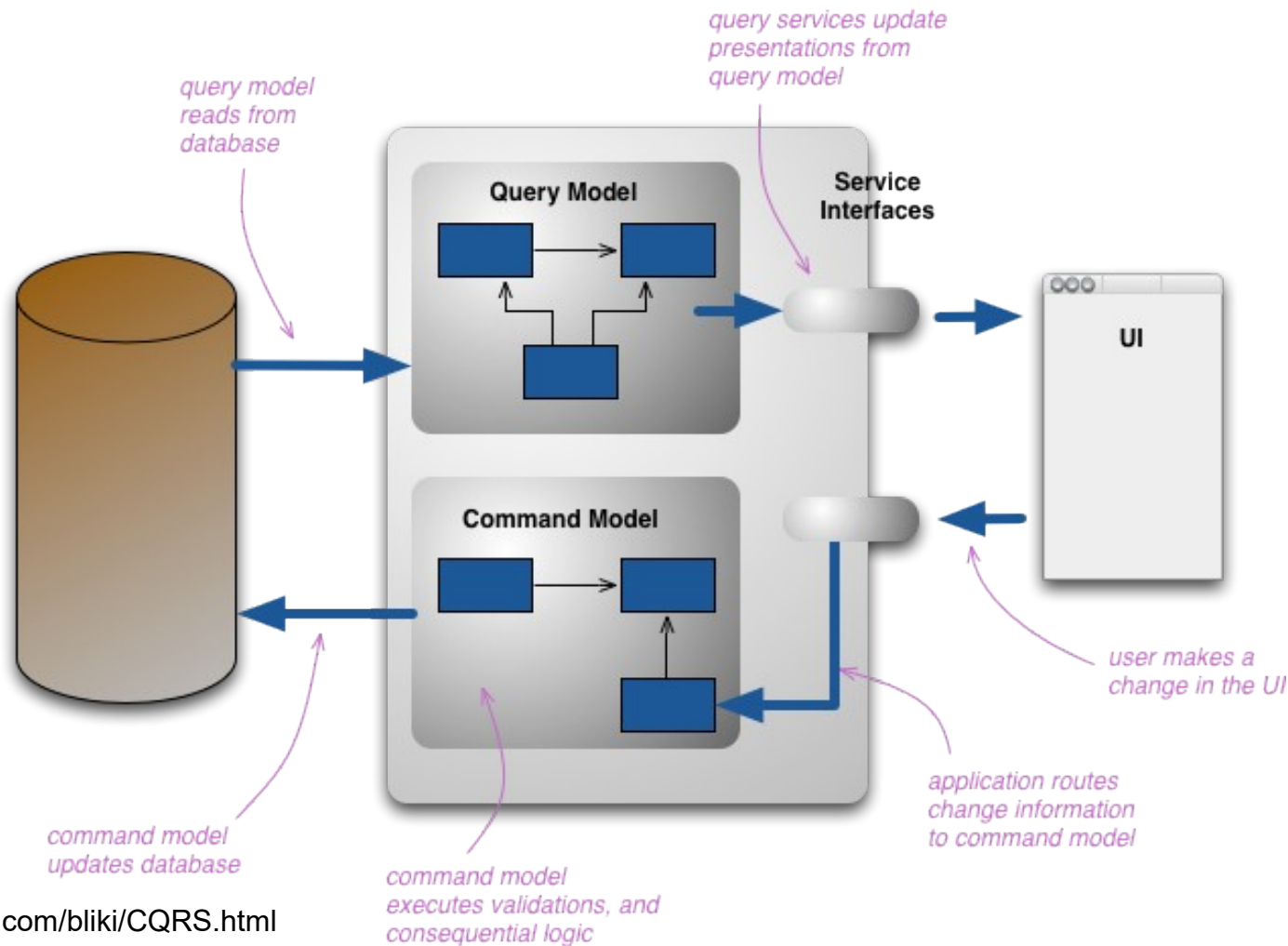
Command Query Responsibility Segregation

Problem:
Gemischtes Modell
für Ansicht
und
Manipulation



Quelle: <http://martinfowler.com/bliki/CQRS.html>

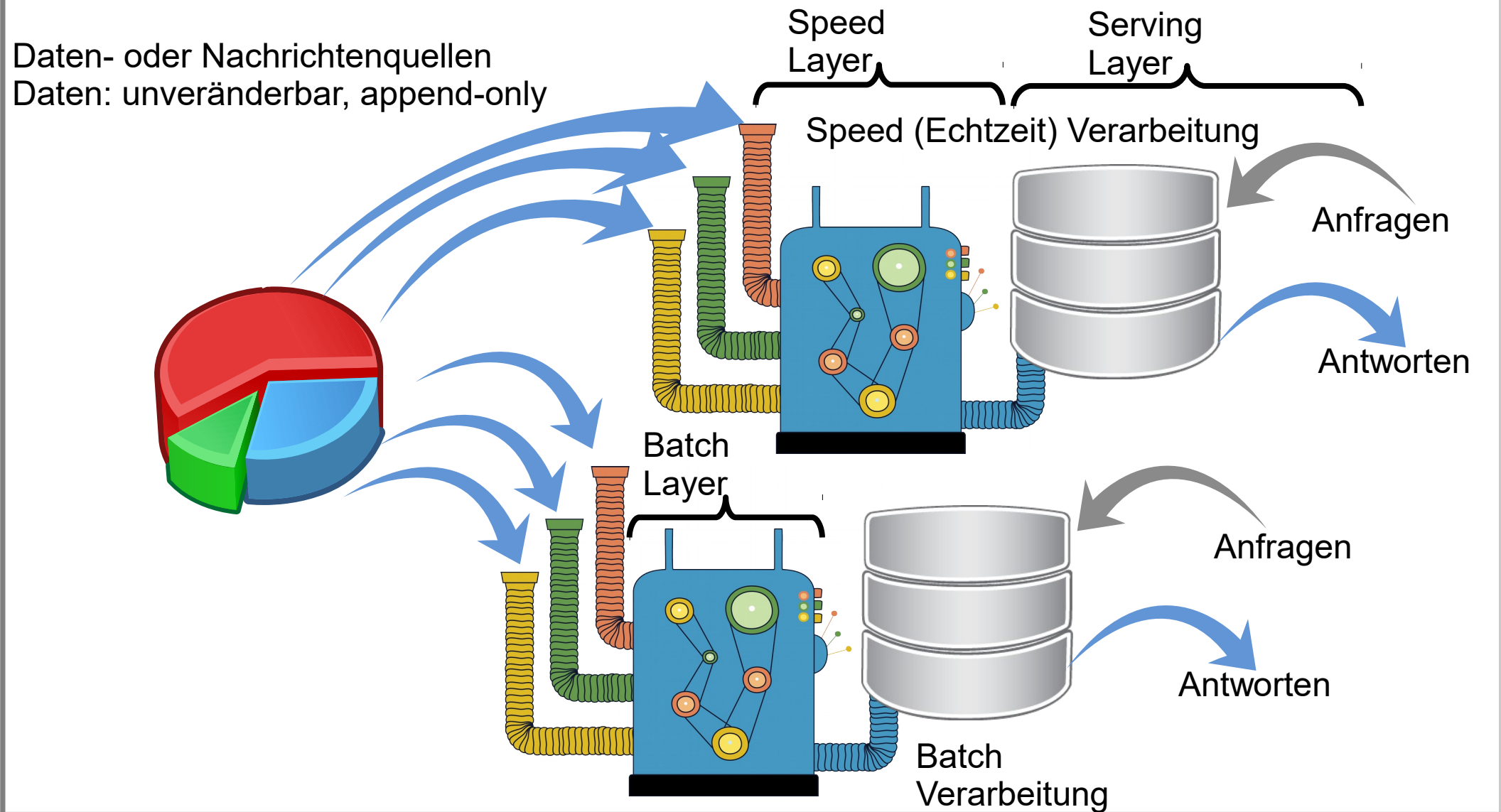
Lösung: Trennung von Sicht- und Manipulationsbefehlen



Quelle: <http://martinfowler.com/bliki/CQRS.html>

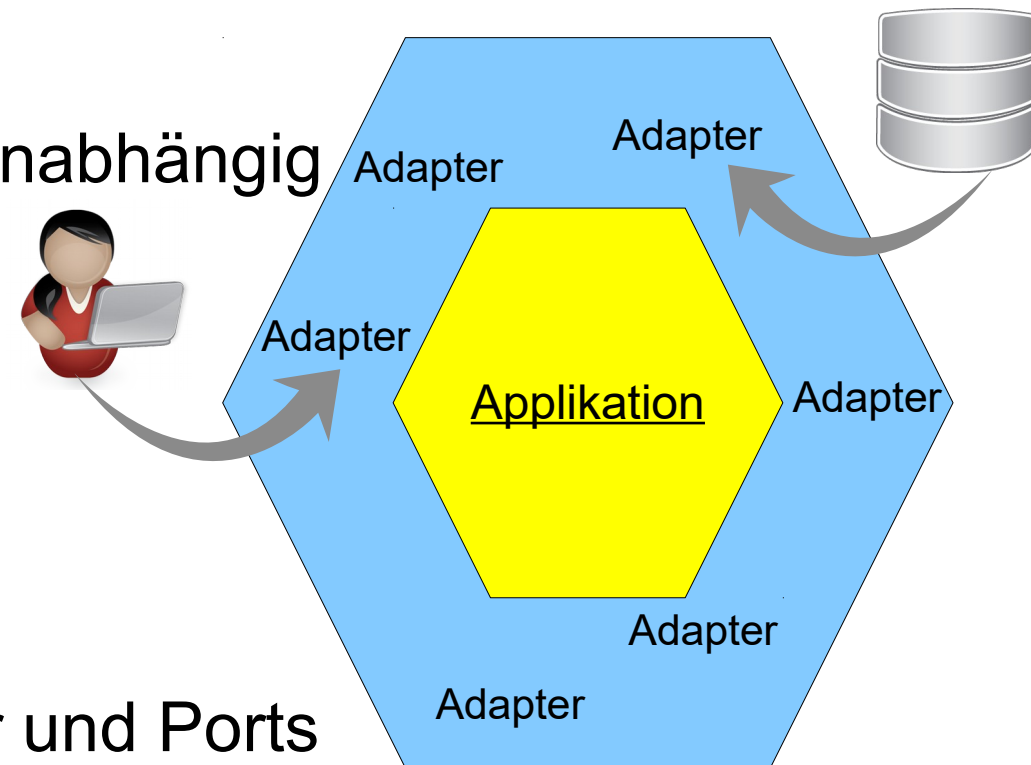
Lambda Architektur verarbeitet großer Datenmengen in Batch- und Stromverarbeitung

- Wichtige Eigenschaften:
 - Unterstützt gleichzeitig akurate Sicht auf Batch-Daten sowie gleichzeitig Stromverarbeitung von Echtzeitdaten (online data)
 - Besitzt nicht Latenzprobleme wie Map-Reduce
 - Datenmodell: append-only, unveränderbar
- Nachteil:
 - Inhärente Komplexität und geringe Verbreitung

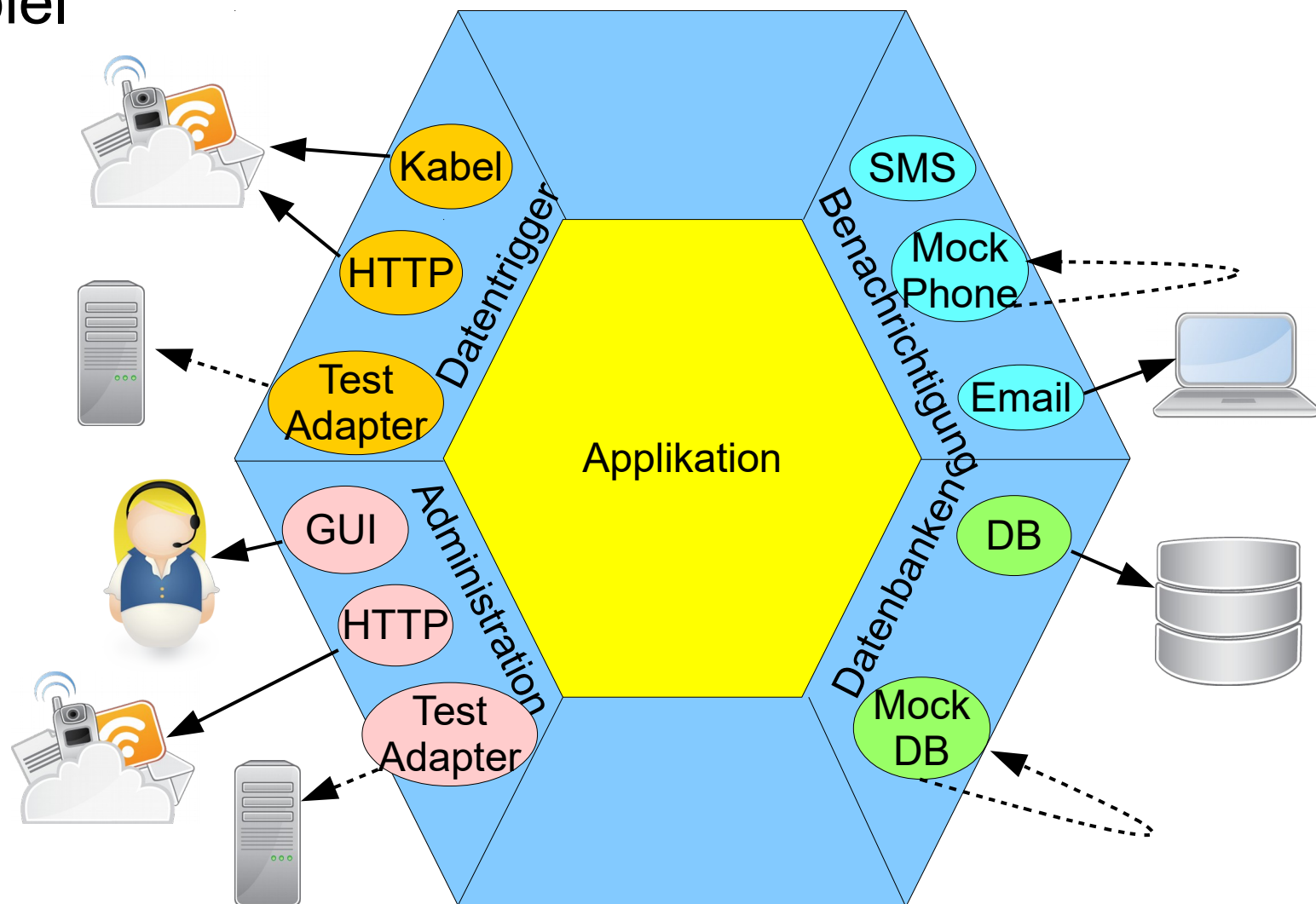


Hexagonal Architekturen unabhängig von Benutzern, Programmen, automatischen Tests und Geräten.

- Wichtige Eigenschaften:
 - Entwicklung und Testing unabhängig von Laufzeitgeräten oder Datenbanken möglich
 - Ports und Adapter Lösung
- Nachteile:
 - Performanzverlust durch Kommunikation via Adapter und Ports



Beispiel



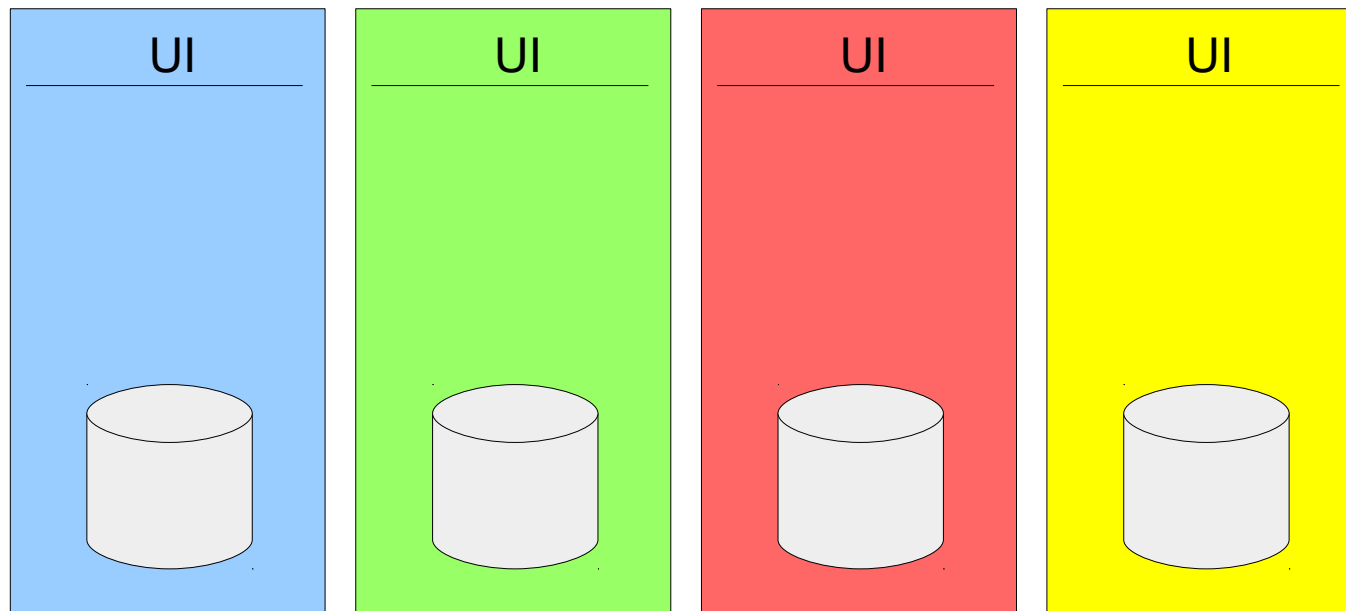
Skalierbare Architekturen

Beispiele für geforderte Skalierbarkeit von Softwarelösungen?

Beispiele für geforderte Skalierbarkeit von Softwarelösungen?

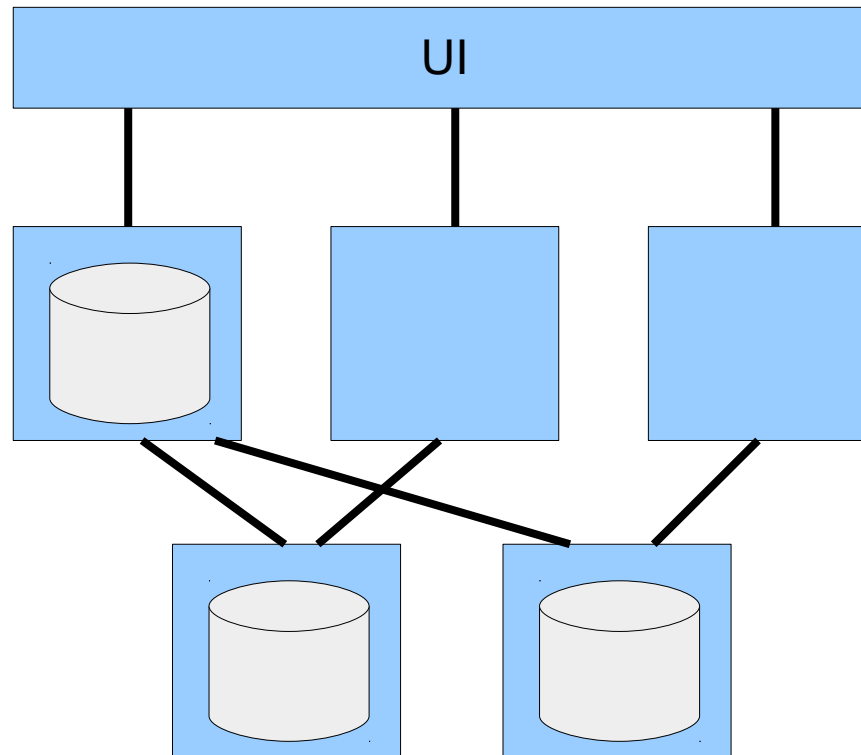
Erwartete gesteigerter Kapazitätsbedarf, z.B. mehr Kunden

Anstelle einer Anwendung, in der alle Features in **einen Prozess** gepackt werden, zerlege Anwendung von vornherein in **mehrere**, voneinander möglichst **unabhängige** Anwendungen. (Auch oft Partition genannt)



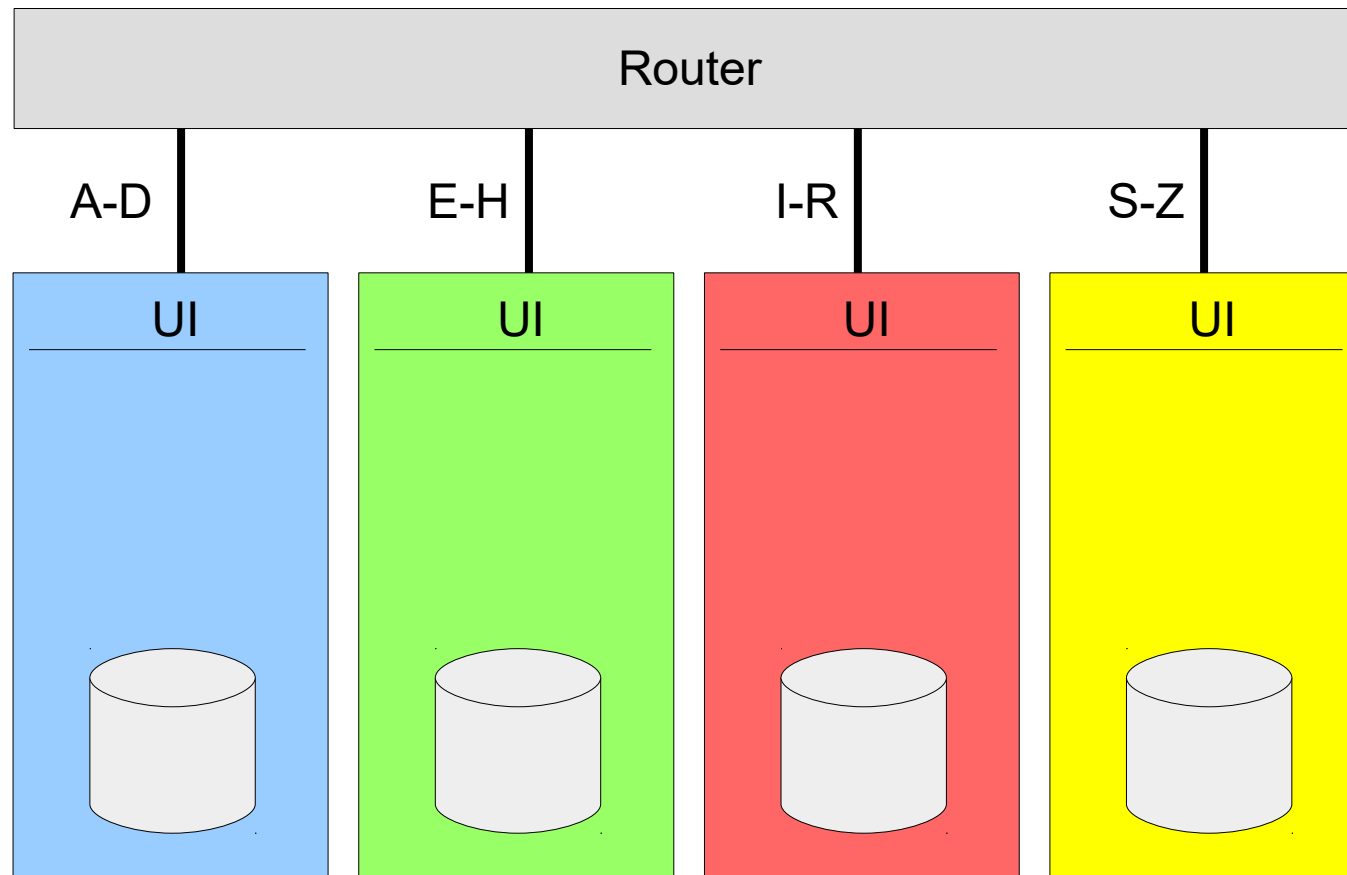
Vertikale Schnitte reichen u.U. nicht aus → daher auch horizontale Schnitte

Kommunikation z.B. über REST (Representational state transfer)

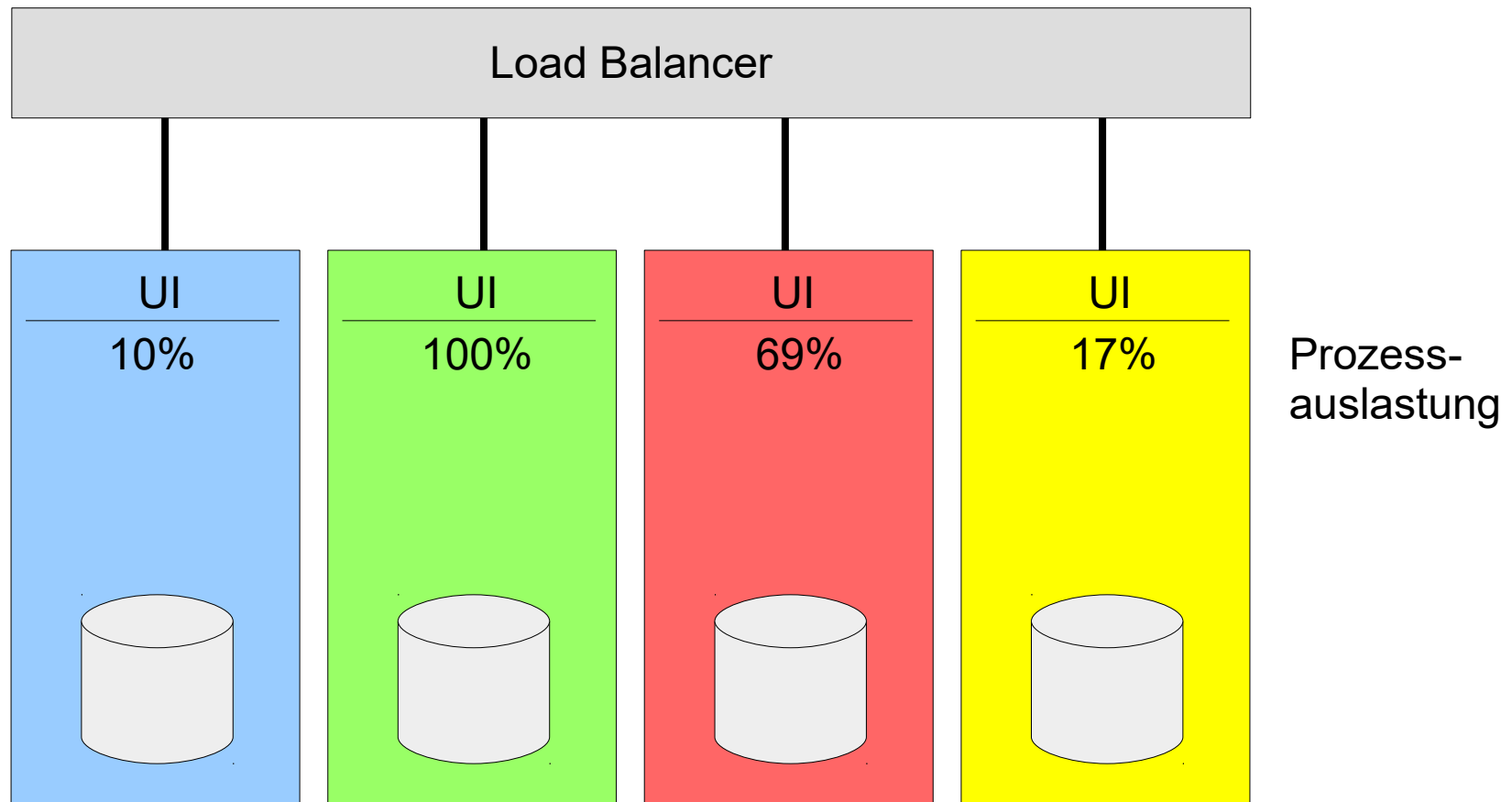


Aufteilung der Eingaben auf dedizierte Prozesse

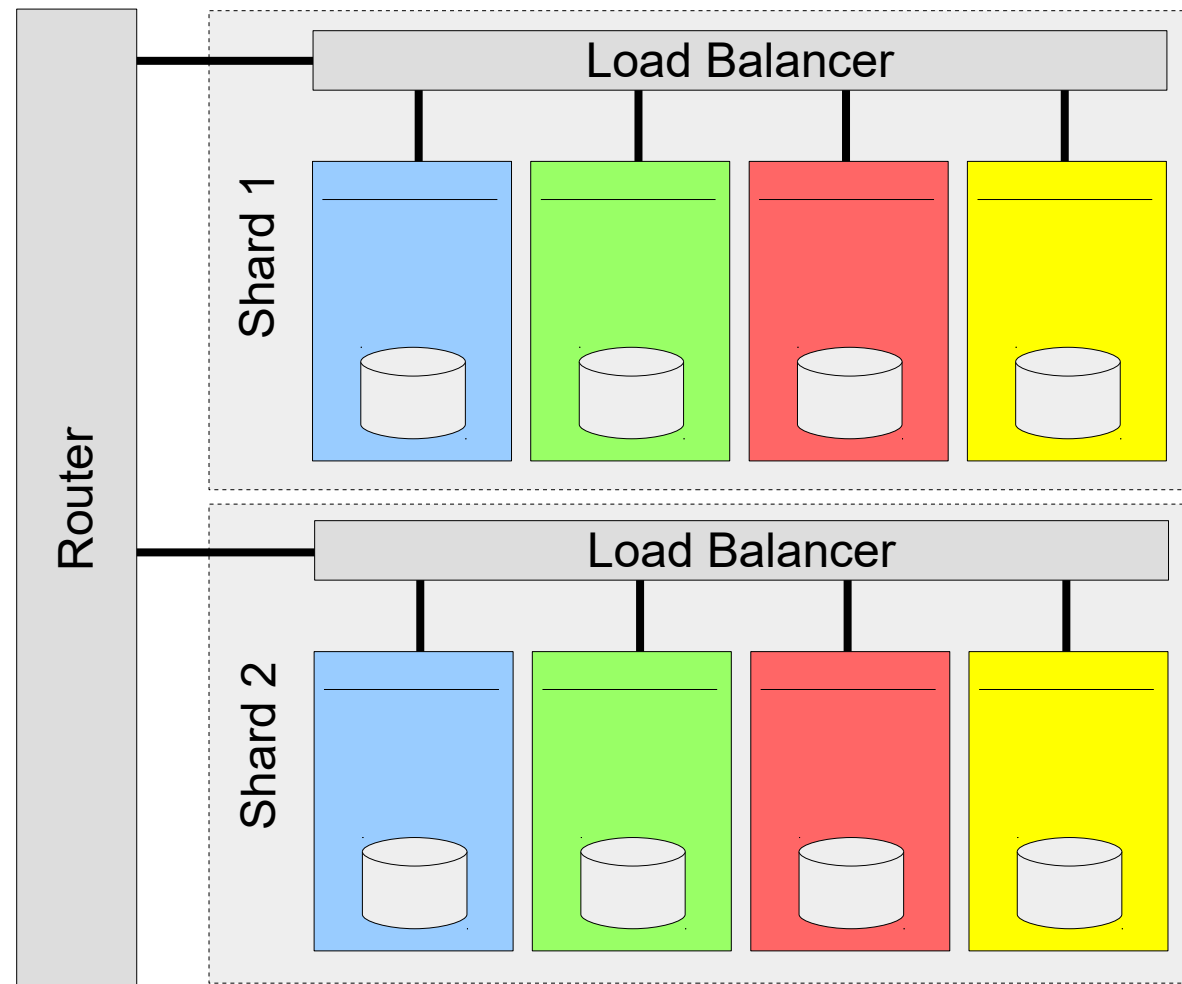
Beispiel: Aufteilung nach Anfangsbuchstaben



Anstatt Sharding mit festem Kriterium, z.B. Anfangsbuchstaben → dynamische Zuordnung der Bearbeitung, z.B. nach Auslastung



Varianten können kombiniert werden um Skalierbarkeit zu erhöhen



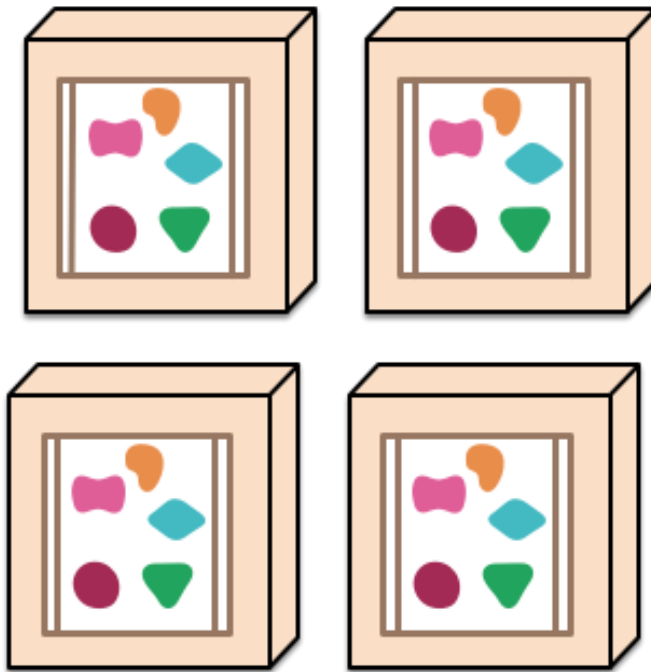
In Microservice Architekturen sind Softwareapplikationen eine Folge von unabhängig einsetzbare (deployable) Diensten

- Wichtige Eigenschaften:
 - Komponentenzerlegung via Dienste leicht möglich
 - Organisiert um betriebswirtschaftliche Funktionen
 - Entwicklung eher Produkte als Projekte
 - Smarte Endpunkte und dumme Verbindungen
 - Ausfallsicher und Evolutionsfähig
 - Dezentrale Anwendungs- und Datenverwaltung

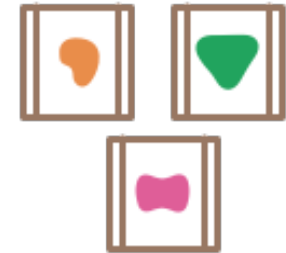
A monolithic application puts all its functionality into a single process...



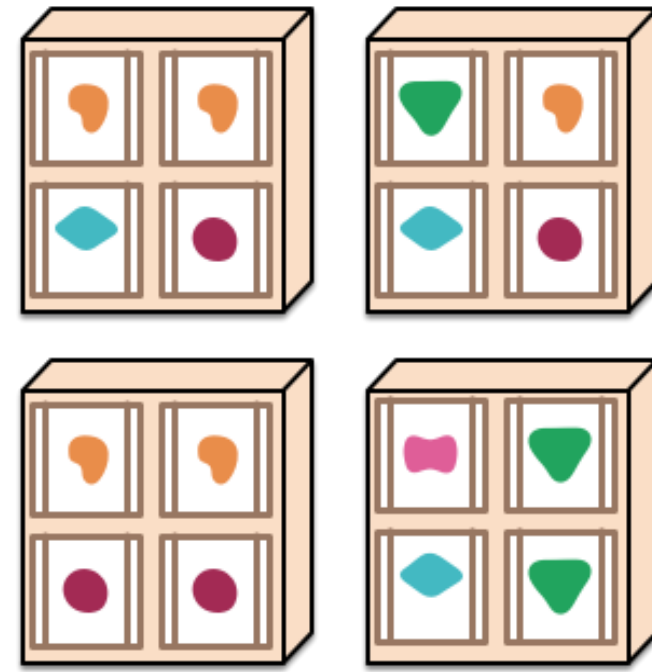
... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



... and scales by distributing these services across servers, replicating as needed.



Quelle: <http://martinfowler.com/articles/microservices.html>

- Verschiedene **Sichten (Views)** auf Mengen von zusammenhängenden Entwurfsentscheidungen für Belange möglich
- Ein **Gesichtspunkt (Viewpoint)** definiert eine Perspektive auf diese Sichten
- Beispiele für Viewpoints:
 - Logischer Gesichtspunkt
 - Physischer Gesichtspunkt
 - Dispositiver Gesichtspunkt (Deployment)
 - Nebenläufiger Gesichtspunkt
 - Verhaltensgesichtspunkt (behaviorial)

Entwurfsprinzipien

Nennen Sie einige
Entwurfsprinzipien!

- **Separation of Concerns** = Separate Belange gehören in separate Module
- **Single Responsibility principle** = jede Klasse nur eine einzige Verantwortung
- **Open-Closed principle** = Module offen für Erweiterungen, geschlossen gegen Änderungen
- **Interface-Segregation principle** = viele Client-spezifische Schnittstellen besser als große Mehrzweckschnittstelle
- **Dependency Inversion principle** = Abhängigkeiten von konkreteren Modulen niedriger Ebenen zu abstrakten Modulen höherer Ebenen gerichtet (Hollywood-principle: don't call us, well call you.).

- **Law of Demeter** (Principle of Least Knowledge) = Objekte sollen nur mit Objekten aus unmittelbaren Umgebung kommunizieren. Keine Komponente soll interne Details anderer Komponenten kennen.
- **Design by Contract** = formale Verträge zur Verwendung von Schnittstellen (Vor-, Nachbedingungen und Invarianten)
- **Don't repeat yourself** (DRY) = Keine unnötigen Codewiederholungen, Absichten nur an einem Ort
- **Self-documentation** = alle Informationen zu einem Modul in diesem selbst enthalten
- **Design for change** = Antizipiere realistische Änderungen

Technische Schulden (technical debt)

- Warum gute, erweiterbare Software? Warum die Kosten?
- **Aber:** Wir müssen für schlechte Entscheidungen später zahlen!
- Zusatzaufwand für Änderungen und Erweiterungen um schlechte Software in gute Software umzuwandeln
- Beispiele
 - Aufschieben oder Nicht-Aktualisierung technischer und fachlicher Softwaredokumentation
 - Aufschieben, Verzicht oder ungenügende Umsetzung automatisierter Modultests und Regressionstests
 - Versäumen der Korrektur von zu großem oder zu komplexen Code und Design

Gründe für technische Schulden

- Fachlicher Druck
- Ungenügende qualitätssichernde Prozesse
- Ungenügendes technisches Wissen
- Ungenügende Kommunikation der gewählten technischen Lösungen und ihrer Hintergründe
- Parallele Entwicklung in eigenen Branches
- Aufgeschobenes Refactoring

„Organizations which design systems [...] are constrained to produce designs which are copies of the communication structures of these organizations.“

Melvin E. Conway: How Do Committees Invent?. In: F. D. Thompson Publications, Inc. (Hrsg.): Datamation. 14, Nr. 5, April 1968, S. 28–31

- Notationen für Softwarearchitekturen wurden vorgestellt (UML)
- Wichtige Entwurfsprinzipien sind Kohäsion und Kopplung
- Verschiedene Architekturstile
- Zusätzliche generelle Entwurfsprinzipien für OO und SA